

基于全量连续权重和差分进化的农作物最优种植策略

摘要

华北山区某乡村有露天耕地 1201 亩及 20 个大棚，常年温度偏低，多数耕地一年只能种一季。不同地块适合种的作物不同，同一地块不能连续重茬，每块地三年内至少得种一次豆类来养地。如何在这些约束条件下合理分配耕地资源，使 2024–2030 年的种植收益最大化，对保障该乡村农业经济可持续发展具有实际意义，也可为类似地区的种植决策提供参考。这是一个典型的多约束组合优化问题。

针对问题 1，在参数相对 2023 年保持稳定的假设下，把超量滞销和超量半价两种情形合并成一个含系数 κ 的分段线性目标函数。约束包括六组：地块–季次–作物结构约束；水浇地单双季种植约束；大棚种植模式约束；跨年不重茬约束；三年豆类轮作约束；地块完整种植约束。采用全量连续权重差分进化求解，使用 1062 个连续权重为决策变量，得到 7 年最优总收益分别为 3929.28 万元和 6224.07 万元，且满足所有约束。

针对问题 2，在问题 1 的约束框架基础上引入各种农作物的四个指标的不确定性变化，包括预期销售量，亩产量，种植成本和销售价格。约束集依旧不变，目标则从确定性收益转为期望收益。各作物波动独立，用 Gauss–Legendre 求积对每个作物算期望，在均匀、三角、正态和 Logit–正态四种分布假设下求解。得到 7 年期望收益 3982–4011 万元，极差不足 1%，表明最优种植方案对各种分布假设具有稳健性，抗风险能力较强。

针对问题 3，在问题 2 的独立波动基础上引入两类 Pearson 相关结构。第一类是同一作物内部预期销售量、销售价格与种植成本之间的块内相关，其中销量与价格的 Pearson 相关系数为 0.4、销量与成本的相关系数为 0.3、价格与成本的相关系数为 0.2。第二类是不同作物之间的块间相关，可替代作物如小麦与玉米的销量呈负相关、相关系数介于负 0.4 至负 0.2 之间，互补作物如大白菜与萝卜等蔬菜的销量呈正相关、相关系数介于 0.25 至 0.3 之间，亩产量波动保持独立（与问题 2 口径一致）。两类相关性共同构成每作物销量、价格、成本三个变量共 123 维的联合分布。采用蒙特卡洛模拟生成相关波动样本（搜索层 $N=2000$ ，评估层 $N=20000$ ）：由相关矩阵经特征分解产生服从联合分布的随机冲击，对每组样本计算对应收益，以样本均值作为期望收益的估计，得到 7 年期望总收益 4032.19 万元，标准差约 41 万元，5% 条件风险价值即 CVaR 约 3948 万元。四种分布假设下期望收益极差约 29 万元、不足 1%，表明方案对分布假设稳健。

最后进行了灵敏度分析，并对模型做了评价和推广。

关键词：全量连续权重；差分进化；蒙特卡洛模拟；Pearson 相关系数

一、 问题重述

1.1 问题背景

华北山区某乡村拥有露天耕地 1201 亩（平旱地、梯田、山坡地、水浇地四种类型，分散为 34 个地块）以及 16 个普通大棚和 4 个智慧大棚（每个 0.6 亩），共计 20 个大棚。该地区常年温度偏低，大多数耕地每年只能种植一季农作物。在乡村振兴战略背景下，如何充分利用有限的耕地资源、因地制宜发展有机种植产业，对乡村经济的可持续发展 [17] 具有重要的现实意义。然而，农作物种植面临多重约束：不同地块类型适宜种植的作物种类不同；同一地块不能连续重茬种植；每块地三年内至少需种植一次豆类作物以利于土壤改良。同时，预期销售量、亩产量、种植成本和销售价格等经济参数存在不同程度的不确定性，增加了决策难度。如何在满足上述约束条件下制定 2024–2030 年的最优种植策略，是一个典型的多约束组合优化问题。

1.2 问题重述

问题 1：已知 2023 年各类农作物的种植面积、亩产量、种植成本、销售价格和预期销售量（以 2023 年产量近似），以及各地块的面积、类型和适宜种植的作物种类。决策变量为每年每季每地块每作物的种植面积。模型任务是在满足不重茬、三年豆类轮作、水浇地模式联动等约束条件下，以 7 年总收益最大为目标，分别针对超量滞销和超量半价两种情形，给出 2024–2030 年的最优种植方案。

问题 2：在问题 1 的约束框架基础上，已知小麦和玉米的预期销售量年增长 5%–10%，其他作物 $\pm 5\%$ ；亩产量受气候影响 $\pm 10\%$ ；种植成本年增 5%；粮食价格基本稳定、蔬菜价格年增 5%、食用菌价格年降 1%–5%（羊肚菌年降 5%）。决策变量和约束条件与问题 1 一致。模型任务是在不确定性条件下以 7 年期望收益最大为目标，给出最优种植方案。

问题 3：在问题 2 的基础上，已知作物间存在可替代性和互补性，预期销售量与销售价格、种植成本之间存在相关性。模型任务是在问题 2 的约束框架基础上，引入作物间的相关结构，通过蒙特卡洛模拟数据驱动求解，给出 2024–2030 年的最优种植策略，并与问题 2 的结果进行比较分析。

二、 问题分析

2.1 问题 1 的分析

问题 1 的核心是在确定性参数条件下，对 20 个地块类型共 82 个决策单元在 41 种作物中进行 7 年种植面积分配，使总收益最大。54 个物理地块按季次展开：平旱地、梯田、山坡地各 1 季，水浇地 2 季，普通大棚 2 季，智慧大棚 2 季。数据方面，附件 1 给出了 34 个地块的面积、类型和可种作物集合，附件 2 给出了 41 种作物的亩产量、种

植成本、销售价格和 2023 年种植情况。关键处理是将 2023 年产量作为预期销售量的近似，题目默认 2023 年全部售出。

约束条件可归纳为六组。地块-季次-作物结构约束：地块-季次-作物的支持集限制。水浇地单双季联动约束：第一季选水稻则单季、第一季含蔬菜则第二季必须选根菜。大棚种植模式约束：普通大棚一季蔬菜一季食用菌、智慧大棚两季蔬菜。跨年不重茬约束：相邻两茬作物集合不相交。三年豆类轮作约束：滚动窗口内每块地至少一季种豆。地块完整性约束：每年 54 个地块都有安排。

目标函数为 7 年总收益最大化，收益核算需考虑超产部分的处理方式，即可选择超量滞销或超量半价两种情形，两种情形可统一为含结算系数的分段线性形式。

难点在于三个方面。一是水浇地模式与作物选择耦合，第一季选水稻则第二季强制休耕。二是不重茬约束使相邻年份的种植决策相互依赖。三是三年豆类轮作约束为滚动窗口形式，需全局规划。

2.2 问题 2 的分析

问题 2 在问题 1 的约束框架基础上引入四类不确定性。预期销售量波动方面，小麦和玉米年增 5% 至 10%，其他作物正负 5%。亩产量受气候影响波动正负 10%。种植成本年增 5%。销售价格分化方面，粮食基本稳定、蔬菜年增 5%、食用菌年降 1% 至 5%。

模型继承关系为约束集不变，目标函数从确定性收益转为期望收益。不确定性进入收益函数的乘性结构，各参数乘以随机波动因子，使得目标函数变为高维积分。

求解策略上，由于各作物的波动可视为独立，问题 2 未考虑相关性，期望收益可通过 Gauss-Legendre 数值求积精确计算每个作物的期望积分（价格因子独立分离后，数值上退化为对产量、销量的二维求积）。

2.3 问题 3 的分析

问题 3 在问题 2 基础上引入两类新因素。第一类是作物间的可替代性，如小麦与玉米、大豆之间存在种植面积的竞争关系，价升则彼销量降（作物种植面积的供应响应与市场趋势见 [4,5,18]）。第二类是作物间的互补性，如土豆与西红柿、大白菜与萝卜等家常配菜组合的销量呈正相关（需求侧协同；豆类仅作为三年轮作硬约束，不涉产量联动）。同时，销量、价格、成本之间存在跨作物相关性。题目明确要求通过模拟数据进行求解，因此采用蒙特卡洛模拟：由多元正态联合分布生成多组相关波动样本，对每组样本计算收益，以样本均值作为期望收益的估计。种植策略需更注重作物组合的分散化，以降低相关性带来的系统性风险。

三、 模型假设

基于题目条件和实际农业生产规律，作出以下假设：

- (1) **产量比例售出假设**：当某作物总产量超过预期销售量时，各产区按产量比例同比例售出，即非滞销比例 $r_{t,j} = \min\{1, D_j/Y_{t,j}\}$ 。该假设保证超产部分在各产区均匀分摊，避免单一产区承担全部滞销损失。
- (2) **独立波动假设**（问题 2）：各类农作物的预期销售量、亩产量、种植成本和销售价格的年度波动相互独立，不考虑作物间的相关性。该假设简化了期望收益的计算（可对各作物独立求积），在问题 3 中被放宽。
- (3) **面积可分假设**：各地块的种植面积可以连续分割，同一地块同一季节可合种不同作物，面积系数 $\alpha_{p,s,j} \in [0, 1]$ 且同季之和不超过 1。该假设使决策变量为连续实数，便于差分进化算法在连续空间中高效搜索。
- (4) **需求侧相关假设**（问题 3）：作物间的可替代性体现为消费者”二选一购买”、互补性体现为”一起购买搭配”，二者均作用于预期销售量的跨作物相关。设替代对集 $\mathcal{S}$ 与互补对集 $\mathcal{C}$ ，销量冲击的相关系数满足

$$\text{Corr}(\xi_a^d, \xi_b^d) = \rho_{ab}, \quad \begin{cases} \rho_{ab} < 0, & (a, b) \in \mathcal{S} \text{ (替代: 销量此消彼长)} \\ \rho_{ab} > 0, & (a, b) \in \mathcal{C} \text{ (互补: 销量同生同息)} \end{cases} \quad (1)$$

且仅销量分量跨作物相关、价格与成本分量跨作物不相关；单作物内部销量、价格、成本存在块内相关（ $\rho_{dp}=0.4$ 、 $\rho_{dc}=0.3$ 、 $\rho_{pc}=0.2$ ）。该假设把题目中的定性关系词转化为可计算的相关系数结构。

- (5) **截断正态联合分布假设**（问题 3）：123 维相关波动向量服从以 $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ 为参数的多元正态分布，各分量支撑区间沿用问题 2 的作物级波动区间 $[lo, hi]$ ，取 $\mu = (lo+hi)/2$ 、 $3\sigma = hi - lo$ 并在采样后钳制回区间；亩产量波动 ξ^q 保持独立，不参与相关结构。

四、符号说明

表 1: 主要符号说明

符号	说明	单位
p	物理地块编号	—
s	季次（1 或 2）	—
j	作物编号（1–41）	—
t	年份（2024–2030）	—
A_p	地块 p 的面积	亩
v	决策向量（全量连续权重编码）， $v \in \mathbb{R}^{1062}$	—

符号	说明	单位
$v_{p,s,j}$	地块 p 第 s 季种植作物 j 的连续权重 (v 的分量), $v_{p,s,j} > 0$ 表示选择	—
D	决策变量维数, $D = \sum_{(p,s)} \mathcal{J}_{p,s} = 1062$	—
$\mathcal{D}$	解码映射: 权重向量 $v \rightarrow$ 面积系数 α (修复链)	—
$F_t(v)$	第 t 年虚拟收益 (DE 搜索层适应度)	元
$P_t(v)$	虚拟罚项 (仅引导搜索方向, 不进入真实收益)	元
$\alpha_{p,s,j}$	面积系数 (解码后), $\sum_j \alpha_{p,s,j} \leq 1$	—
$q_{u,j}$	决策单元 u 种植作物 j 的亩产量 ("地块-作物" 档位取值)	斤/亩
$c_{u,j}$	单元 u 种植作物 j 的单位面积种植成本 ("地块-作物" 档位取值)	元/亩
$p_{u,j}$	单元 u 种植作物 j 的销售价格 ("地块-作物" 档位取值)	元/斤
D_j	作物 j 的预期销售量	斤
u	决策单元 (由物理地块 p 与季次 s 展开)	—
A_u	决策单元 u 的面积	亩
$x_{t,u,j}$	第 t 年单元 u 上作物 j 的实际种植面积	亩
$Y_{t,j}$	第 t 年作物 j 的总产量, $Y_{t,j} = \sum_u q_{u,j} x_{t,u,j}$	斤
$G_{t,j}$	第 t 年作物 j 的总产值, $G_{t,j} = \sum_u p_{u,j} q_{u,j} x_{t,u,j}$ (各档位单价分别计入)	元
$C_{t,j}$	第 t 年作物 j 的总成本, $C_{t,j} = \sum_u c_{u,j} x_{t,u,j}$	元
$r_{t,j}$	第 t 年作物 j 的结算比例, $r_{t,j} = \min\{1, D_j/Y_{t,j}\}$ (各档位按产量比例同比例售出)	—
$\psi(r)$	产值结算系数, $\psi(r) = r + \kappa(1 - r)$	—
τ	相对 2023 年的年偏移, $\tau = t - 2023$	—
$\xi^q, \xi^d, \xi^p, \xi^c$	亩产量、预期销售量、销售价格、种植成本的随机波动	—
$\tilde{Y}_{t,j}, \tilde{D}_{t,j}, \tilde{G}_{t,j}, \tilde{C}_{t,j}$	随机总量, = 对应确定性量 $\times$ 波动乘子	—
$\bar{p}_{t,j}$	价格演化期望因子, $\bar{p}_{t,j} = \mathbb{E}[(1 + \xi^p)^\tau]$	—
$\mathcal{S}, \mathcal{C}$	替代对集、互补对集 (问题 3)	—
$\mathbf{R}$	123×123 相关矩阵 (问题 3)	—
κ	超产结算系数 (0= 滞销, 0.5= 半价)	—
Π_t	第 t 年的总收益	元

符号	说明	单位
$\mathbb{E}[\Pi_t]$	第 t 年收益的数学期望	元
$\mathcal{J}_{p,s}$	地块 p 第 s 季的可种作物集合	—
$\mathcal{B}$	豆类作物集合	—
ξ	问题 3 随机冲击向量, $\xi = (\xi^d, \xi^p, \xi^c) \in \mathbb{R}^{123}$ (按作物分块)	—
μ, σ, Σ	多元正态均值向量、标准差向量、协方差矩 阵 (问题 3), $\Sigma = \text{diag}(\sigma)\mathbf{R}\text{diag}(\sigma)$	—
$\Pi_n(x)$	情景 n 下方案 x 的 7 年总收益 (问题 3)	元
$\hat{\mathbb{E}}[\Pi]$	期望收益的样本估计 (SAA), $\hat{\mathbb{E}}[\Pi] =$ $\frac{1}{N} \sum_{n=1}^N \Pi_n(x)$	元
N	蒙特卡洛情景样本数 (搜索层 2000、评估层 20000)	—
$\lambda_1, \lambda_2, \lambda_3, \lambda_4$	虚拟罚项系数 ($10^6, 10^6, 10^6, 50$ 元/作物·季)	—
$n_t^{\text{enc}}, n_t^{\text{re}}, n_t^{\text{bean}}, n_t^{\text{crop}}$	第 t 年编码违规数、重茬违规茬次数、闭合 窗口缺豆地块数、选中作物季数 (虚拟罚项 计数)	—
$S_{t,k}$	第 t 年第 k 茬的作物集合 (不重茬约束)	—
f_j^q, f_j^d, f_j^p	作物 j 亩产量、销量、价格波动的概率密度 函数 (问题 2)	—
Ω_j	作物 j 的积分域, $[lo_j^q, hi_j^q] \times [lo_j^d, hi_j^d] \times$ $[lo_j^p, hi_j^p]$	—
$\tilde{r}_{t,j}$	问题 2 随机结算比例, $\tilde{r}_{t,j} =$ $\min\{1, \tilde{D}_{t,j}/\tilde{Y}_{t,j}\}$	—
$\tilde{r}_{t,j,n}$	情景 n 下结算比例, $\tilde{r}_{t,j,n} =$ $\min\{1, \tilde{D}_{t,j,n}/\tilde{Y}_{t,j,n}\}$	—
n_{quad}	高斯-勒让德求积阶数 (问题 2, $n_{\text{quad}}=16$ 已 收敛)	—
$\text{CVaR}_{5\%}$	5% 条件风险价值 (最坏 5% 情景的平均收 益)	元

五、模型的建立与求解

5.1 数据预处理

附件数据包含乡村的现有耕地、经济参数明细、经济数据和 2023 年种植情况四张表，均来自赛题附件 1/附件 2（经整理存于 data/目录），字段刻画与用途见表 2，需进行以下预处理。

表 2: 参考数据的来源、字段刻画与用途

数据表（来源）	主要字段（单位）	规模	在模型中的用途
乡村的现有耕地（附件 1）	地块编号、名称、面积（亩）、可种植作物编号	34 个地块	决策单元展开、面积 A_p 、支持集 $\mathcal{J}_{p,s}$
经济参数明细（附件 2）	地块类型、种植季次、作物编号、亩产量（斤/亩）、种植成本（元/亩）、售价区间与中值（元/斤）	10 档 $\times$ 41 作物	档位参数 $q_{u,j}, c_{u,j}, p_{u,j}$
经济数据（附件 2）	作物编号、2023 种植面积（亩）、2023 产量（斤，即销售量）、销量/产量/成本/价格年变化率	41 种作物	销量基准 D_j 、波动区间（问题 2/3）
2023 种植情况（附件 2）	种植地块、种植季次、作物编号、种植面积（亩）	87 条记录	2023 基线（重茬/豆类窗口输入）

地块—单元展开。34 个物理地块中，平旱地、梯田、山坡地各 1 季，水浇地 2 季、普通大棚 2 季、智慧大棚 2 季。按季次展开后得到 82 个决策单元，每个单元独立对应一组可种植作物集合和支持面积。地块编号规则为：单季地块直接使用地块编号，双季地块追加后缀“-1”和“-2”分别表示第一季和第二季。

预期销售量推算。题目未给出 2024 年及以后的预期销售量，合理假设以 2023 年实际产量作为预期销售量的近似基准。根据 2023 年种植情况表和经济参数明细表中的亩产量数据，对每种作物计算 2023 年总产量：将各地块 2023 年该作物的种植面积乘以对应地块类型和季次下的亩产量，汇总得到该作物的年度总产量。该总产量即为后续各年的预期销售量基准值。

地块类型映射。将 34 个物理地块按首字母分类：A 类为平旱地、B 类为梯田、C 类为山坡地、D 类为水浇地、E 类为普通大棚、F 类为智慧大棚。每类地块的可种植作物集合由附件 1 中“可种植作物编号”字段给出，以分号分隔。

波动参数提取。从经济数据表中提取四类波动参数：预期销售量年变化率、亩产量年变化率、种植成本年变化率、销售价格年变化率。其中预期销售量和销售价格以区间形式给出（如“5% 至 10%”），需解析为上下界。种植成本统一取年增 5%。

相关结构构建（问题 3）。构建 123 维相关矩阵，其中块内相关反映同一作物销量、价格、成本之间的联动关系，块间相关反映可替代作物间的竞争效应和互补作物间的协同效应。相关矩阵经特征值截断修正半正定后，因含近似零特征值导致 Cholesky 分解数值不稳定，故改用特征分解（谱分解）生成相关波动样本。

5.2 问题 1：确定性最优种植模型

5.2.1 决策变量

采用全量连续权重编码，把一年种植方案表示为一个 D 维连续向量 v 。向量分量按”（物理地块 p ，季次 s ）”分块排列，块内为该季次支持集 $\mathcal{J}_{p,s}$ 的作物槽：

$$v = [v^{(p,s)}]_{\forall (p,s)}, \quad v^{(p,s)} = (v_{p,s,j})_{j \in \mathcal{J}_{p,s}}, \quad v_{p,s,j} \in \mathbb{R} \quad (2)$$

$v_{p,s,j} > 0$ 表示选择作物 j 并分配面积、 $v_{p,s,j} \leq 0$ 表示不选择——用连续变量的”有无”代替离散 0-1 选择，无需”作物 ID+ 面积”双基因块。变量总数

$$D = \sum_{(p,s)} |\mathcal{J}_{p,s}| = 1062 \quad (3)$$

即 82 个决策单元（54 个物理地块按季次展开）允许作物数之和，离散变量为零。

解码器把任意权重向量 v 映射为结构合法的面积系数 $\alpha_{p,s,j}$ ：

$$\alpha_{p,s,j} = \mathcal{D}(v)_{p,s,j} \in [0, 1], \quad \sum_{j \in \mathcal{J}_{p,s}} \alpha_{p,s,j} \leq 1 \quad (4)$$

实际种植面积 $x_{t,u,j} = \alpha_{p,s,j} \cdot A_u$ （ u 为由 (p,s) 展开的决策单元）。 $\mathcal{D}$ 由修复链实现（负值裁剪、重茬权重置零、 $\sum \alpha$ 归一化、水浇地模式裁决等，细节见求解算法一节），保证任意 v 解码后均为结构合法方案；年际方案再经编码器/解码器与 YearPlan 结构（式 (9)）双向映射。

5.2.2 目标函数

记 t 为年份（2024–2030），7 年总收益最大化。经济参数（亩产量 q 、单位面积成本 c 、售价 p ）均按”地块–作物”档位粒度取值（同一类型–季节档位内取值相同，不同档位取值不同），收益按档位分别核算：产值与成本沿档位累加，仅超产滞销判断基于作物级预期销量。统一收益模型为：

$$\max \sum_{t=2024}^{2030} \sum_{j=1}^{41} [G_{t,j} \psi(r_{t,j}) - C_{t,j}] \quad (5)$$

其中：

- 作物级总产量： $Y_{t,j} = \sum_u q_{u,j} \cdot x_{t,u,j}$ ， $q_{u,j}$ 为单元 u 种植作物 j 的亩产量；

- 作物 j 总产值: $G_{t,j} = \sum_u p_{u,j} q_{u,j} x_{t,u,j}$, $p_{u,j}$ 为单元 u 上作物 j 的档位售价 (各档位单价分别计入产值, 不合并为作物级均价);
- 作物 j 总成本: $C_{t,j} = \sum_u c_{u,j} x_{t,u,j}$, $c_{u,j}$ 为对应档位单位面积成本;
- 非滞销/总产比: $r_{t,j} = \min\{1, D_j/Y_{t,j}\}$ (作物级 D_j , 各档位按产量比例同比例售出);
- 结算函数: $\psi(r) = r + \kappa(1 - r)$, $\kappa = 0$ (超量滞销) 或 $\kappa = 0.5$ (超量半价)。

当 $Y_{t,j} \leq D_j$ 时, $r = 1$, $\psi = 1$, 收入为 $G_{t,j}$ (全价售出); 当 $Y_{t,j} > D_j$ 时, $r = D_j/Y_{t,j}$, 收入为 $G_{t,j}[r + \kappa(1 - r)]$ (超量部分按各档位产值比例分摊, 并按 κ 系数结算)。该“产量-需求截断”的收益核算结构即报童模型 (newsvendor) 的形式化推广 [14]。

5.2.3 约束体系

六组约束的数学表述如下:

A 结构约束:

$$\alpha_{p,s,j} \geq 0, \quad \sum_{j \in \mathcal{J}_{p,s}} \alpha_{p,s,j} \leq 1, \quad \forall (p, s, j) \quad (6)$$

B 水浇地联动约束 (对水浇地 d):

- 若第一季选水稻 ($v_{d,1,\text{rice}} > 0$): 第二季必须为空, $\sum_j \alpha_{d,2,j} = 0$;
- 若第一季含蔬菜 ($\exists j \in \text{蔬菜}, \alpha_{d,1,j} > 0$): 第二季必须选恰好一种根菜 (白萝卜/红萝卜/大白菜之一)。

C 大棚模式约束: 普通大棚 E 第一季蔬菜、第二季食用菌; 智慧大棚 F 两季均为蔬菜。由支持集 $\mathcal{J}_{p,s}$ 保证。

D 不重茬约束: 对每块地 p , 将相邻两年各季排成时间序列 $S_{t-1,1} \rightarrow S_{t-1,2} \rightarrow S_{t,1} \rightarrow S_{t,2}$, 要求相邻两茬作物集合不相交:

$$S_{t-1,k} \cap S_{t,k'} = \emptyset, \quad \text{相邻两茬} \quad (7)$$

E 三年豆类轮作约束: 设豆类集合 $\mathcal{B} = \{1, \dots, 5, 17, 18, 19\}$, 滚动窗口 $[w, w + 2]$ 内每块地至少一季种豆 (豆类-禾谷轮作对产量、营养与收入的增益已有实证 [6,7,8]):

$$\exists y \in [w, w + 2], S_y \cap \mathcal{B} \neq \emptyset, \quad \forall \text{地块} \quad (8)$$

F 完整性约束: 每年 54 个物理地块均有安排 (空地地块视为休耕)。

5.2.4 求解算法：全量连续权重差分进化

方案的数据结构（统一中间表示）。全年种植方案定义为”地块 $\rightarrow$ 季次 $\rightarrow$ （作物，面积系数）”的嵌套映射：

$$\text{YearPlan} = \{p \mapsto \{s \mapsto [(j, \alpha_{p,s,j})]\}\}, \quad 0 \leq \alpha_{p,s,j} \leq 1, \quad \sum_j \alpha_{p,s,j} \leq 1 \quad (9)$$

其中空列表表示休耕，实际种植面积 $x_{t,u,j} = \alpha_{p,s,j} A_u$ ；7 年方案为 $\{2024, \dots, 2030\} \mapsto \text{YearPlan}$ 的年度字典。该结构同时被贪心构造、DE 解码、局部搜索、规则硬校验与收益核算五个环节使用，是求解器的统一中间表示。YearPlan 与式 (2) 定义的决策向量 v 通过编码器（encode）与解码器（decode）双向映射：选中作物的面积系数写入对应权重槽、未选中为 0。

解码修复链。 decode 按固定顺序把任意权重向量映射为结构合法的方案：负值裁剪（ $\tilde{v}_j = \max\{0, v_j\}$ ）；重茬排除（相邻茬集合作物的权重置零）；水浇地隐式模式切换（第一季水稻权重 > 0.02 则判为单季水稻模式，否则水稻权重置零）；面积归一化（各季 $\sum_j \alpha > 1$ 时等比缩放至 1， < 1 时允许部分种植或休耕）；水浇地模式修复（水稻与蔬菜互斥、单季水稻清空第二季、第二季不能独立存在、第一季含蔬菜则补最优根菜）；大棚塌缩修复（第一季被清空而第二季仍有作物时，重投影合法第一季）；补齐 54 个地块。修复链保证任意 v 解码后均为结构合法方案，再经硬校验确认零违规后计算纯收益。

算法流程。 采用 DE/rand/1/bin 策略 [9]（ $F=0.7$ 、 $CR=0.9$ 、 $NP=32$ 、 $G=80$ ），7 年方案按年度逐年求解：

1. **贪心热启动：**对每一年，按全价边际利润 $p_{u,j}q_{u,j} - c_{u,j}$ （档位参数）降序逐地块填入作物，面积系数 $\alpha = \min\{\text{剩余面积系数}, (D_j - \text{已产量})/(q_{u,j}A_u)\}$ （需求耗尽后转入超量种植），避开上一茬作物，生成满足大部分约束的初始方案；
2. **低维预热：**每年先用 $K=2$ 槽位编码（每地块-季次至多 2 个作物槽，共约 296 维）作短预算 DE，得到结构化好解后编码为全量权重向量，作为当年全量 DE 的热启动种子；
3. **可行邻域播种：**以热启动种子为种群个体 0，其余个体对非零权重作 ± 0.15 抖动并随机注入 1-3 个作物槽，构成 $NP=32$ 的初始种群；对闭合豆类窗口内缺豆地块，逐个个体强制种豆；
4. **DE 进化：**逐个体解码后以**虚拟收益**为适应度：

$$F_t(v) = \Pi_t(\text{decode}(v)) - P_t(v), \quad P_t(v) = \lambda_1 n_t^{\text{enc}}(v) + \lambda_2 n_t^{\text{re}}(v) + \lambda_3 n_t^{\text{bean}}(v) + \lambda_4 n_t^{\text{crops}}(v) \quad (10)$$

其中 Π_t 为第 t 年真实收益（问题 1 为确定性收益、问题 2/3 为期望收益）， n^{enc} 为编码规则违规数（ $\alpha < 0$ 、 $\sum \alpha > 1$ 、水浇地模式冲突等）、 n^{re} 为重茬违规茬

次数、 n^{bean} 为本年闭合豆类窗口缺豆地块数、 n^{crops} 为选中作物季数（稀疏引导项）；罚项系数 $\lambda_1 = \lambda_2 = \lambda_3 = 10^6$ 、 $\lambda_4 = 50$ 元/作物·季。罚项 P_t 是虚拟量，只存在于搜索引导层（决定 DE 个体优劣与搜索方向），解码修复链与硬校验保证最终方案零违规， P_t 不进入真实收益——最终结果为硬校验通过后的纯收益。按 DE/rand/1/bin 变异交叉、贪心选择迭代 $G=80$ 代，取最优个体解码存入当年方案；

5. 局部搜索与硬校验：对每年每地块试候选方案（有效边际 top-N、水浇地模式、休耕），接受虚拟收益提高者，轮间收敛即停；随后补做豆类修复兜底，最终方案经 RuleChecker 硬校验（违规收集并报错）确认零违规后，计算纯收益作为结果。

5.2.5 求解结果与分析

表 3: 问题 1 求解结果

情形	7 年总收益（万元）	编码违规	重茬违规	豆类违规
(1) 超量滞销	3929.28	0	0	0
(2) 超量半价	6224.07	0	0	0

情形（1）7 年总收益为 3929.28 万元，年均约 561 万元；情形（2）为 6224.07 万元，年均约 889 万元。情形（2）收益显著更高，源于允许超量半价销售使超产部分仍可获得 50% 的收入。复种指数分别为 1.04 和 1.10，后者接近全满（水浇地双季充分利用）。

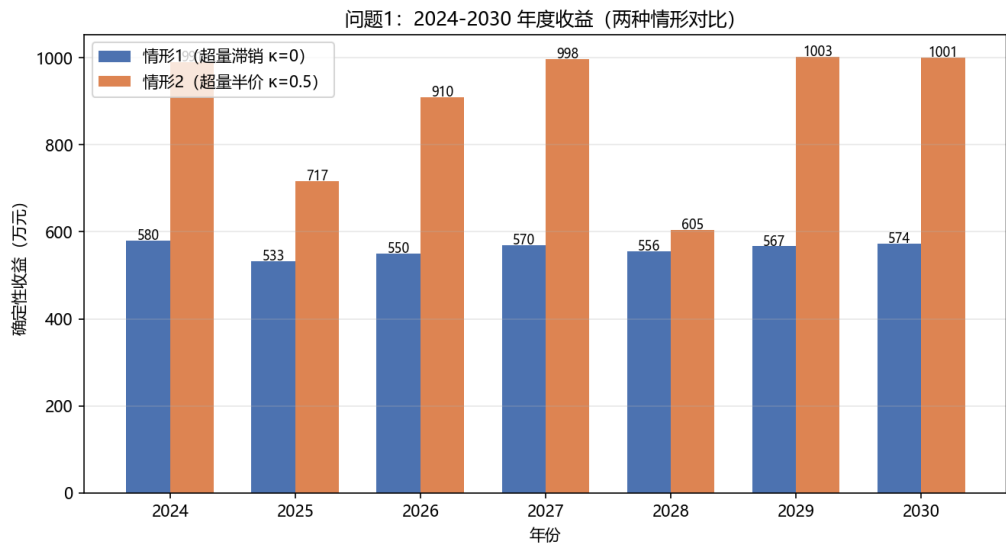


图 1: 问题 1 各年度收益对比（情形 1 vs 情形 2）

由图 1 可知，两种情形的年度收益均围绕年均水平波动，并未单调上升——问题 1 为确定性静态参数模型（经济参数相对 2023 年保持不变），年度间差异主要源于重茬、

豆类轮作等跨年约束导致的被动结构调整。情形 2（超量半价）各年收益均显著高于情形 1（超量滞销），平均高出约 330 万元/年，差距最大出现在 2029 年（约 436 万元）；2025、2028 年两种情形收益同步回调，系当年种植结构受跨年约束调整所致。

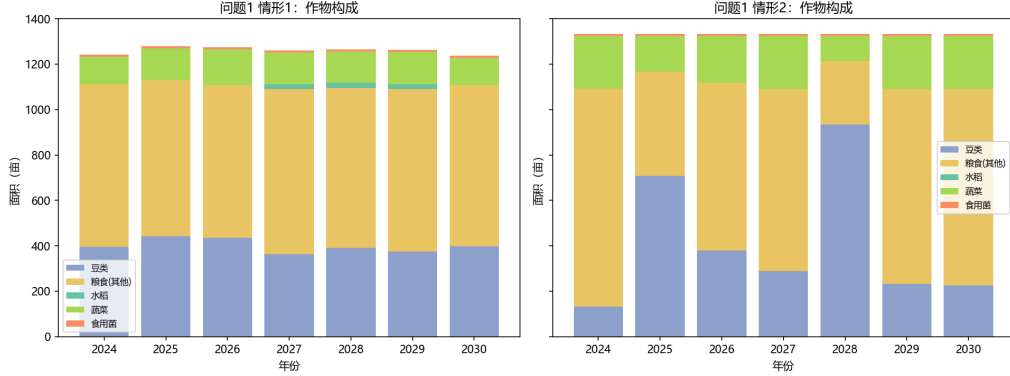


图 2: 问题 1 情形 2 各年度作物种植面积构成

由图 2 可知，情形 2 的作物种植面积在 7 年间保持相对稳定，粮食作物（小麦、玉米、大豆等）占据主要面积，蔬菜和食用菌在大棚地块中维持固定比例。值得注意的是，豆类作物的种植面积呈小幅波动，这与三年豆类轮作约束的滚动窗口效应有关：某些年份需集中安排豆类种植以满足约束，随后年份可适当减少。整体复种指数稳定在 1.10 左右，表明水浇地双季种植模式得到充分利用。

5.3 问题 2：不确定性下的期望收益模型

5.3.1 不确定性建模

在问题 1 的约束框架 $\mathcal{C}_1$ 基础上，引入四类随机波动。记 t 为年份（2024–2030）， $\tau = t - 2023$ 为相对 2023 年的年偏移。经济参数按“地块–作物”档位粒度取值，确定性总量（与问题 1 口径一致，各档位单价/成本分别计入）为：

$$Y_{t,j} = \sum_u q_{u,j} x_{t,u,j}, \quad G_{t,j} = \sum_u p_{u,j} q_{u,j} x_{t,u,j}, \quad C_{t,j} = \sum_u c_{u,j} x_{t,u,j} \quad (11)$$

其中 $q_{u,j}$ 、 $p_{u,j}$ 、 $c_{u,j}$ 分别为决策单元 u 种植作物 j 的亩产量、售价与单位面积成本（“地块–作物”档位取值）， $x_{t,u,j}$ 为第 t 年单元 u 上作物 j 的实际种植面积（由决策权重解码得到）。 D_j 为作物 j 的基准预期销售量（以 2023 年产量近似）。波动作用于作物级随机乘子，随机总量为：

$$\tilde{Y}_{t,j} = (1 + \xi_{t,j}^q)^\tau Y_{t,j} \quad (\text{随机总产量, } \xi^q \text{ 为亩产量波动}) \quad (12)$$

$$\tilde{D}_{t,j} = (1 + \xi_{t,j}^d)^\tau D_j \quad (\text{随机销售量, } \xi^d \text{ 为销量波动}) \quad (13)$$

$$\tilde{G}_{t,j} = (1 + \xi_{t,j}^p)^\tau G_{t,j} \quad (\text{随机总产值, } \xi^p \text{ 为价格波动}) \quad (14)$$

$$\tilde{C}_{t,j} = (1 + 0.05)^\tau C_{t,j} \quad (\text{随机总成本, 成本年增 5\%}) \quad (15)$$

其中 ξ 为区间 $[l_o, h_i]$ 上的随机波动，服从均匀、三角、正态或 Logit–正态分布之一。

5.3.2 期望收益计算

记 Π_t 为第 t 年的总收益，目标函数转为 7 年期望收益之和最大化：

$$\max \sum_{t=2024}^{2030} \mathbb{E}[\Pi_t] \quad (16)$$

超产滞销判断基于作物级预期销量 D_j （各档位按产量比例同比例售出），随机结算比例为：

$$\tilde{r}_{t,j} = \min \left\{ 1, \tilde{D}_{t,j} / \tilde{Y}_{t,j} \right\} \quad (17)$$

由于各作物的四类波动相互独立，期望收益可逐作物分解。收益核算采用与问题 1 统一的分段结算式：售出部分按全价、超产部分按系数 κ 结算，其中 $(x)_+ = \max\{x, 0\}$ 。对作物 j ，任一波动实现 (ξ^q, ξ^d, ξ^p) 下的实际收益等于该实现下的全价产值 $G_{t,j}(1 + \xi^q)^\tau(1 + \xi^p)^\tau$ （ $= \sum_u p_{u,j} q_{u,j} x_{t,u,j}(1 + \xi^q)^\tau(1 + \xi^p)^\tau$ ，售价、亩产量与面积沿档位累加，产量与价格乘子均计入）乘以结算函数 $\tilde{r}_{t,j} + \kappa(1 - \tilde{r}_{t,j})_+$ 。设 f_j^q 、 f_j^d 、 f_j^p 分别为作物 j 亩产量、预期销售量、销售价格波动的概率密度函数（均匀、三角、正态或 Logit-正态分布之一 [16]，支撑区间 $[lo_j^q, hi_j^q]$ 、 $[lo_j^d, hi_j^d]$ 、 $[lo_j^p, hi_j^p]$ 按作物给定），作物 j 第 t 年的期望收益为三重积分：

$$\mathbb{E}[\Pi_{t,j}] = \iiint_{\Omega_j} G_{t,j}(1 + \xi^q)^\tau(1 + \xi^p)^\tau \left[\tilde{r}_{t,j} + \kappa(1 - \tilde{r}_{t,j})_+ \right] f_j^q(\xi^q) f_j^d(\xi^d) f_j^p(\xi^p) d\xi^q d\xi^d d\xi^p \quad (18)$$

其中 $\Omega_j = [lo_j^q, hi_j^q] \times [lo_j^d, hi_j^d] \times [lo_j^p, hi_j^p]$ 为作物 j 的波动支撑区间， $\tilde{r}_{t,j} = \min\{1, \tilde{D}_{t,j} / \tilde{Y}_{t,j}\}$ 为式 (17) 定义的结算比例（随 ξ^q, ξ^d 变化）。由于价格波动 ξ^p 独立且仅以乘子进入积分，其期望因子 $\bar{p}_{t,j} = \mathbb{E}[(1 + \xi^p)^\tau]$ 可提出积分号——粮食、蔬菜、羊肚菌的价格年变化为确定性常数（期望因子分别为 1、1.05 $^\tau$ 、0.95 $^\tau$ ），食用菌为一维数值积分——数值实现中三重积分随之退化为对 (ξ^q, ξ^d) 的二维求积。7 年各作物期望收益加总为：

$$\mathbb{E}[\Pi_t] = \sum_{j=1}^{41} \left[\mathbb{E}[\Pi_{t,j}] - \tilde{C}_{t,j} \right] \quad (19)$$

其中 $\kappa = 0$ （超量滞销）时超产部分不计收入， $\kappa = 0.5$ （超量半价）时按半价结算，问题 2 与问题 3 均取 $\kappa = 0$ ，与问题 1 情形 1 口径一致，保证三问可比。数值上，对每个作物的二维积分采用 Gauss-Legendre 求积近似（ $n=16$ ，经 16/24/40 三档验证结果差异小于 0.01 万元），计算复杂度为 $O(41 \times n^2)$ ，远低于蒙特卡洛模拟的 $O(N \times 41)$ 次收益计算；求积精度达小数点后四位且无随机误差，可为 DE 搜索提供精确的适应度评估。

5.3.3 求解结果与分析

表 4: 问题 2 四种分布假设下的期望收益

分布	期望收益（万元）	编码违规	重茬违规	豆类违规
均匀分布	3988.74	0	0	0
三角分布	3981.76	0	0	0
正态分布	3994.55	0	0	0
Logit-正态分布	4010.62	0	0	0

四种分布假设下期望收益的极差为 $4010.62 - 3981.76 = 28.86$ 万元,相对极差仅 0.72%,表明最优种植方案对分布假设稳健,具有较强的抗风险能力。

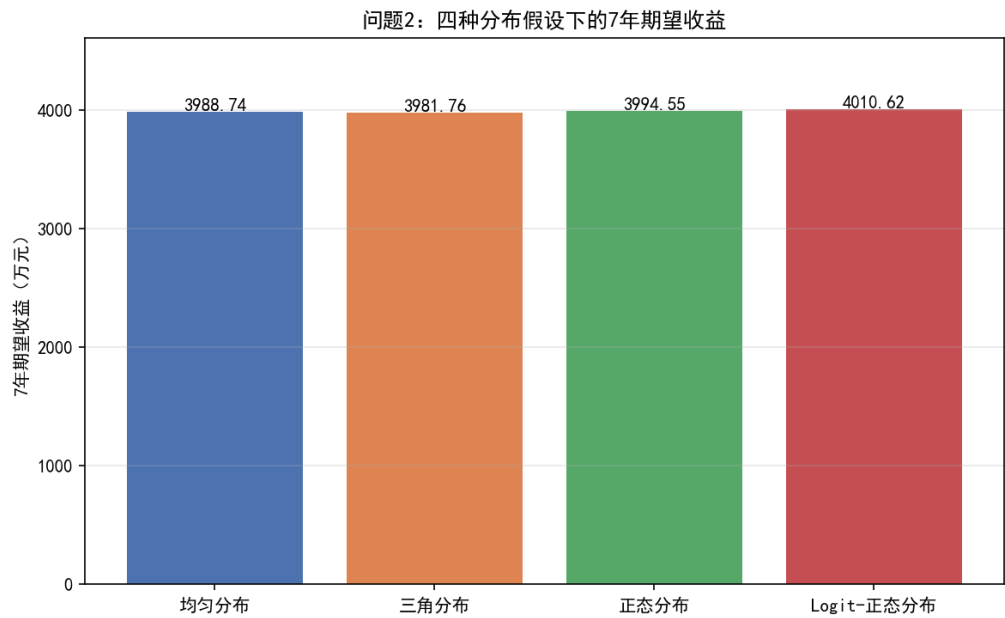


图 3: 问题 2 四种分布假设下的期望收益对比

由图 3 可知,四种分布假设下的期望收益差异极小,柱状图高度几乎持平。Logit-正态分布的期望收益略高(4010.62 万元),三角分布略低(3981.76 万元),两者相差仅 28.86 万元,相对极差不足 1%。这一结果表明:在独立波动假设下,分布类型的选择对最优种植方案的影响可忽略不计。其原因在于,各作物的波动区间较窄(如销量正负 5%、价格年变化多在正负 5% 以内),不同分布的尾部差异在小波动范围内被稀释,期望收益主要由波动区间的中点决定。

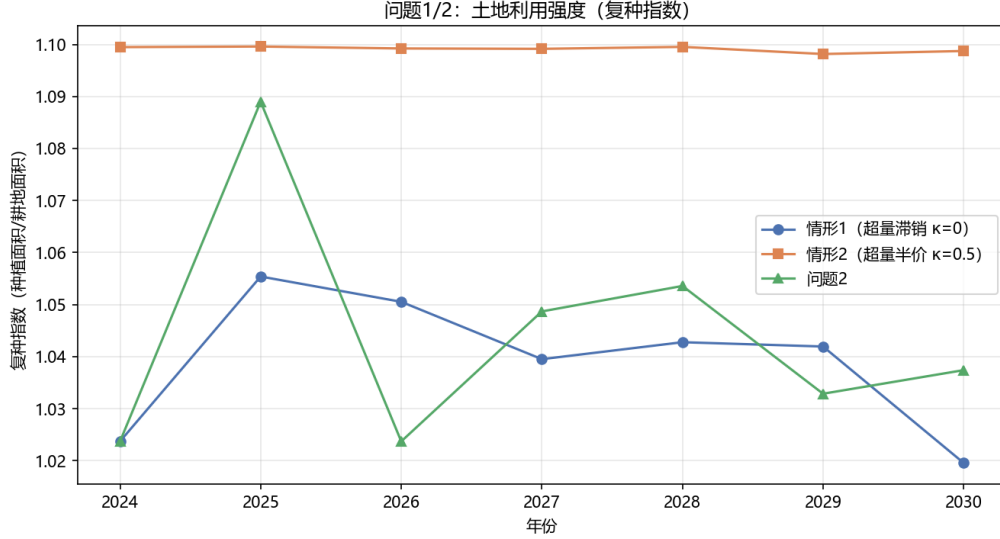


图 4: 问题 1 两种情形与问题 2 的耕地利用率对比

由图 4 可知，问题 1 情形 1 与问题 2 的复种指数在各年度基本一致（均值均约 1.04），说明不确定性因素的引入并未显著改变耕地的利用模式，最优方案在确定性和随机性条件下均能有效利用可用耕地资源；问题 1 情形 2 因允许超量半价销售，复种指数稳定在约 1.10。问题 2 的利用率在部分年份略低于情形 1，这是因为随机波动使得方案在边际地块上选择更保守的种植策略，以降低不确定性带来的风险。

5.4 问题 3：相关性条件下的蒙特卡洛模拟模型

5.4.1 相关结构建模

在问题 2 的独立波动基础上引入作物间的相关性。设年度随机向量 $\xi = (\xi^d, \xi^p, \xi^c) \in \mathbb{R}^{123}$ ，按作物分组排列：作物 j 对应销量、价格、成本三个分量 $(\xi_j^d, \xi_j^p, \xi_j^c)$ 。亩产量波动 ξ^q 保持独立（与问题 2 口径一致），不参与相关结构。令 ξ 服从多元正态联合分布（相关正态采样见文献 [11]）：

$$\xi \sim \mathcal{N}(\mu, \Sigma), \quad \Sigma = \text{diag}(\sigma) \mathbf{R} \text{diag}(\sigma) \quad (20)$$

其中 μ 、 σ 分别为 123 个分量的均值向量与标准差向量， $\mathbf{R}$ 为 123×123 相关矩阵，由文献证据 [10,15] 和数据驱动确定：

- **块内相关**：同一作物内部销量、价格、成本之间的联动，取 $\rho_{dp}=0.4$ 、 $\rho_{dc}=0.3$ 、 $\rho_{pc}=0.2$ ；
- **块间相关（可替代性）**：61 对替代作物销量负相关，如小麦与玉米 $\rho = -0.40$ 、小麦与水稻 $\rho = -0.20$ 、白萝卜与红萝卜 $\rho = -0.30$ 等，强度 $|\rho| \in [0.2, 0.4]$ ；
- **块间相关（互补性）**：6 对互补作物销量正相关，如土豆与西红柿 $\rho=0.25$ 、西红柿与茄子 $\rho=0.30$ 、青椒与辣椒 $\rho=0.30$ 、黄瓜与生菜 $\rho=0.25$ 、大白菜与白/红萝卜 $\rho=0.30$ 。

作物 j 的销量、价格、成本分量在 $\mathbf{R}$ 中按块排列、占据索引 $3(j-1)+1, 3(j-1)+2, 3(j-1)+3$ 。对替代/互补对 (a, b) ，仅在两作物的销量分量处写入相关、其余跨作物元素为 0：

$$R_{3(a-1)+1, 3(b-1)+1} = R_{3(b-1)+1, 3(a-1)+1} = \rho_{ab} \quad (21)$$

其中 $R_{ij} = \text{Corr}(\xi_i, \xi_j)$ 。由于任意指定的相关元素拼接出的矩阵未必半正定，对 $\mathbf{R}$ 做特征值截断修正 [12]（负特征值钳制到 10^{-8} ）后经特征分解构造采样矩阵，保证样本服从指定的联合相关结构。

5.4.2 情景收益与目标函数

对任一波动情景 n ，作物 j 第 t 年的随机总量沿用问题 2 的趋势结构，仅销量、价格、成本引入相关冲击（亩产量独立）：

$$\tilde{Y}_{t,j,n} = Y_{t,j}(1 + \xi_{t,j,n}^q)^\tau, \quad \tilde{G}_{t,j,n} = G_{t,j}(1 + \xi_{t,j,n}^p)^\tau, \quad \tilde{C}_{t,j,n} = C_{t,j} 1.05^\tau (1 + \xi_{t,j,n}^c) \quad (22)$$

$$\tilde{D}_{t,j,n} = D_j \begin{cases} (1 + \xi_{t,j,n}^d)^\tau, & j \in \{\text{小麦, 玉米}\} \\ (1 + \xi_{t,j,n}^d), & \text{其余} \end{cases} \quad (23)$$

其中 $(\xi_n^d, \xi_n^p, \xi_n^c)$ 由式 (20) 的联合分布按情景抽样、 ξ^q 独立 $\pm 10\%$ 波动。情景收益仍按问题 1 的统一滞销式结算（ $\psi(r) = r + \kappa(1 - r)$ ， $\kappa = 0$ ）：

$$\Pi_n(x) = \sum_{t=2024}^{2030} \sum_{j=1}^{41} \left[\tilde{G}_{t,j,n} \psi(\tilde{r}_{t,j,n}) - \tilde{C}_{t,j,n} \right], \quad \tilde{r}_{t,j,n} = \min \left\{ 1, \tilde{D}_{t,j,n} / \tilde{Y}_{t,j,n} \right\} \quad (24)$$

目标为样本均值近似期望（SAA，见文献 [13]），作为 DE 搜索层的适应度：

$$\max_x \hat{\mathbb{E}}[\Pi(x)] = \max_x \frac{1}{N} \sum_{n=1}^N \Pi_n(x) \quad (25)$$

5.4.3 蒙特卡洛模拟求解

赛题要求”通过模拟数据进行求解”。采用蒙特卡洛模拟（ $N = 2000$ ）：

1. 由 $\mathbf{R}$ 经特征分解（谱分解）构造采样矩阵，生成 N 组年度相关波动样本 $(\xi_n^d, \xi_n^p, \xi_n^c)$ ，亩产量波动 ξ^q 独立抽样；
2. 对给定种植方案 x ，计算 N 个情景收益 $\Pi_n(x)$ ；
3. 以样本均值 $\hat{\mathbb{E}}[\Pi] = \frac{1}{N} \sum_n \Pi_n$ 作为适应度；
4. 复用问题 1/2 的 DE 求解框架（黑盒替换为 MC 评估）。

蒙特卡洛模拟的关键参数选择：搜索层取 $N = 2000$ 在计算精度和效率之间取得平衡（MC 收敛测试表明， $N \geq 2000$ 时样本均值相对 $N = 50000$ 的偏差不超过 0.03%），评估层进一步加密至 $N = 20000$ 以给出最终报告值。相关矩阵 $\mathbf{R}$ 经特征值截断修正半正定后，因含近似零特征值导致 Cholesky 分解数值不稳定，故改用特征分解（谱分解）构造采样矩阵，确保生成的样本服从指定的联合分布。块内相关和块间相关共同作用使得收益波动呈现系统性特征：当某类作物价格普遍上涨时，其替代品的销量同步下降，互补品的销量协同上升，这种联动效应在独立假设下无法体现。

5.4.4 求解结果与分析

表 5: 问题 3 求解结果（搜索层 $N=2000$ ，评估层 $N=20000$ ）

评估框架	指标	问题 2 方案	问题 3 方案	差 (p3-p2)
独立求积（问题 2 框架）	7 年期望收益（万元）	3994.55	4004.01	+9.46 (+0.24%)
相关 MC（问题 3 框架）	7 年期望收益（万元）	4022.67	4032.19	+9.52 (+0.24%)
相关 MC	标准差（万元）	40.54	41.11	+0.57
相关 MC	5% CVaR（万元）	3940.08	3948.36	+8.28

问题 3 的 7 年期望总收益为 4032.19 万元。在同一相关 MC 框架下比较，问题 3 方案（4032.19 万元）较问题 2 方案（4022.67 万元）提高 9.52 万元（+0.24%）；在独立求积框架下亦提高 9.46 万元（+0.24%），两个框架结论一致。收益的小幅提升源于考虑相关结构后方案在同类作物间进行更优的分散化配置：替代作物间避免集中种植、互补作物间协同安排。标准差 41.11 万元、5% 条件风险价值（CVaR）为 3948.36 万元，与问题 2 方案（40.54 万元/3940.08 万元）基本持平，CVaR 改善 8.28 万元，表明考虑相关结构未显著增加尾部风险，方案在相关性条件下仍具有较好的风险控制能力。

5.4.5 年度收益对比

表 6: 问题 2 与问题 3 年度期望收益对比

年份	问题 2 方案（相关 MC，万元）	问题 3 方案（相关 MC，万元）	差异（万元）	问题 3
2024	574.79	574.79	0.00	
2025	531.85	534.74	+2.89	
2026	562.77	565.07	+2.30	
2027	584.40	588.01	+3.61	
2028	570.84	566.99	-3.85	
2029	577.76	579.99	+2.23	
2030	620.26	622.60	+2.34	
合计	4022.67	4032.19	+9.52	

各年收益大多小幅提升，其中 2027 年增幅最大（+3.61 万元），2028 年因作物组合调整出现小幅回调（-3.85 万元），7 年合计仍提升 9.52 万元。问题 3 的种植面积在部分年份略有调整（如 2026 年减少 31 亩、2029 年增加 22 亩），体现了相关性驱动下的作物组合分散化策略。

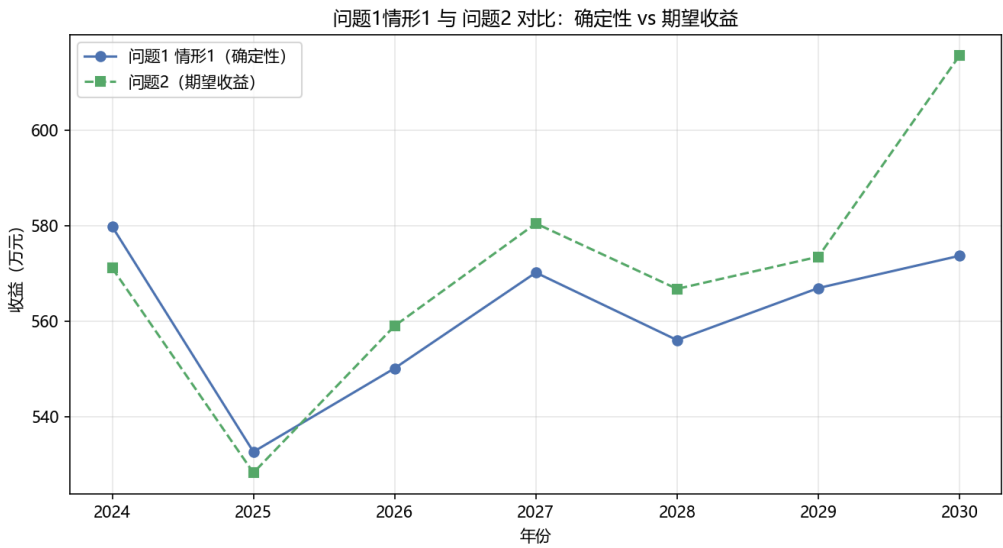


图 5: 问题 1 与问题 2 各年度收益对比

由图 5 可知，问题 2（随机性）的 7 年期望收益合计 3994.55 万元，略高于问题 1 情形 1（确定性、超量滞销）的 3929.28 万元（+1.7%）。这一差异源于价格随机化的凸性效应： $E[(1 + \xi^p)^\tau] > (1 + E\xi^p)^\tau$ （ $\tau \geq 1$ 时复合增长为凸函数），使期望收入略高于确定性基准。值得注意的是，两种框架下的年度收益变化趋势高度一致，均在 2028 年出

现小幅回调后于 2029–2030 年回升，表明年度间的结构性差异（如作物轮作周期）是影响收益走势的主要因素，而非随机波动本身。

六、 结果分析

6.1 约束可行性验证

所有求解结果均通过 `RuleChecker.check_full` 硬校验，验证项包括：结构约束 (A1–A5)、水浇地联动 (B1–B5)、大棚模式 (C)、不重茬 (D)、三年豆类轮作 (E)、完整性 (F)。问题 1、问题 2 和问题 3 的所有方案违规数均为零，模型闭合性得到保证。

6.2 误差分析

蒙特卡洛收敛性（问题 3 相关 MC）。两方案 7 年合计期望收益随样本量 N 的收敛表现见表 7，其中偏差以 $N=50000$ 为参照。

表 7: 问题 3 蒙特卡洛收敛测试 (SE=std/ $\sqrt{N}$, 偏差相对 $N=50000$)

N	问题 2 期望 (万元)	问题 3 期望 (万元)	问题 3 SE (万元)	偏差 (%)
1000	4022.78	4032.48	1.28	+0.01
2000	4022.40	4031.87	0.92	−0.00
5000	4023.57	4033.25	0.59	+0.03
10000	4023.06	4032.57	0.41	+0.02
20000	4022.67	4032.19	0.29	+0.01
50000	4022.41	4031.92	0.18	0.00

$N \geq 2000$ 后两方案期望收益即趋于稳定（相对 $N=50000$ 的偏差不超过 0.03%）； $N=20000$ 时标准误仅 0.29 万元（不足 0.01%）。因此搜索层 $N=2000$ 、评估层 $N=20000$ 的配置下，蒙特卡洛抽样误差可以忽略。

求积阶数（问题 2 框架，正态分布）： $n_{\text{quad}}=16/24/40$ 时，问题 2 方案期望收益分别为 3994.55/3994.56/3994.57 万元，问题 3 方案为 4004.01/4004.02/4004.03 万元，求积误差小于 0.01 万元， $n_{\text{quad}}=16$ 已充分收敛；且求积无随机误差，可为 DE 搜索提供精确的适应度评估。

6.3 分布稳健性分析

问题 2 在四种分布假设下期望收益为 3981.76–4010.62 万元（极差 0.72%）、问题 3 方案为 3991.55–4020.60 万元（极差 29.05 万元、0.7%），最优方案对分布假设稳健，抗风险能力较强。这一结论在更换分布类型（均匀、三角、正态、Logit–正态）后均成立。

6.4 灵敏度分析

超产结算系数 κ : 问题 1 中 κ 从 0（滞销）增至 0.5（半价），总收益从 3929.28 万元增至 6224.07 万元，增幅 58.4%。这表明超产处理方式是影响收益的关键参数。

重茬与豆类轮作约束的代价: 以 2023 年实际种植为参照（年均 592.63 万元），情形 1 的最优解（年均 561.33 万元）存在 5.3% 的差距。诊断表明，该差距完全来自重茬、三年豆类轮作与超量滞销等硬约束的刚性代价：照抄 2023 年种植方案将在 2024 年全面积重茬违规，592.63 万元/年这一”当年产销平衡”的理想状态在连续 7 年的可行域内并不存在；而 DE 相对贪心基线已提升 13.3%（+462 万元），说明解已贴住可行域上界。若允许重茬，水浇地可连续种植高价值蔬菜，收益可进一步上升，但将导致土壤肥力下降，不符合可持续发展理念。

豆类轮作约束: 三年豆类轮作约束限制了部分地块的作物选择自由度，但有利于土壤改良和后作增产，是长期可持续性的保障。

DE 算法稳定性: 对问题 1 进行多随机种子运行：情形 1 在 5 组种子（32×40）下收益为 3916.00–3924.99 万元（极差 0.23%）；情形 2 在 4 组种子（32×80）下收益为 6200.60–6224.07 万元（极差 0.42%），最终取最优种子方案（6224.07 万元），表明算法收敛稳定，多次运行的次优空间小于 0.5%。

相关矩阵参数灵敏度（问题 3）: 对块内相关系数 ρ_{dp} 、 ρ_{dc} 、 ρ_{pc} 及替代/互补幅度进行灵敏度分析。各参数在基准值 $\pm 50\%$ 范围内变化时，期望收益变化均不超过 0.01%（表 8），表明模型对相关矩阵参数不敏感。

表 8: 问题 3 相关矩阵参数灵敏度

参数	取值	期望收益（万元）	Δ （万元）	Δ （%）
基准	—	4032.19	0.00	0.00
ρ_{dp}	0.2	4031.74	−0.45	−0.01
ρ_{dp}	0.6	4032.27	+0.08	0.00
ρ_{dc}	0.15	4031.97	−0.22	−0.01
ρ_{dc}	0.45	4032.47	+0.28	+0.01
ρ_{pc}	0.1	4032.32	+0.13	0.00
ρ_{pc}	0.3	4032.19	−0.00	−0.00
替代幅度	0.5	4032.36	+0.17	0.00
替代幅度	1.5	4031.92	−0.27	−0.01
互补幅度	0.5	4031.74	−0.45	−0.01
互补幅度	1.5	4031.80	−0.39	−0.01

波动区间灵敏度: 对销量、价格、成本、亩产量四类波动区间进行缩放灵敏度分析（表 9）。亩产量波动影响最大（ $\pm 50\%$ 缩放导致收益变化 +0.28%/−0.38%），成本波动影响最小（几乎为零），表明收益对成本参数最不敏感。

表 9: 问题 3 波动区间缩放灵敏度

因素	缩放	问题 2 期望 (万元)	问题 3 期望 (万元)	Δ (%)
基准	1.00	4022.67	4032.19	0.00
销量	0.50	4026.01	4035.28	+0.08
销量	1.50	4017.28	4027.19	-0.12
价格	0.50	4021.35	4030.86	-0.03
价格	1.50	4024.79	4034.32	+0.05
成本	0.50	4022.66	4032.18	-0.00
成本	1.50	4022.68	4032.20	+0.00
亩产量	0.50	4034.47	4043.55	+0.28
亩产量	1.50	4007.13	4016.78	-0.38

6.5 相关结构误设与随机种子稳健性

相关结构误设 (表 10): 若真实数据实际不存在相关性 ($R = I$), 将两方案在独立假设下重新评估。期望收益几乎不变 (差异小于 0.01 万元), 相关结构仅放大风险 (标准差增加约 4 万元、5%CVaR 下降约 7 万元); 即使真实数据无相关, 问题 3 方案也不损失期望收益。这表明最优方案对相关结构误设鲁棒。

表 10: 相关结构误设 ($R = I$ vs R) 下的期望收益、标准差与 5%CVaR (万元)

相关假设	问题 2 期望	问题 2 标准差	问题 2 CVaR5	问题 3 期望	问题 3 标准差	问题 3 CVaR5
$R = I$ (独立)	4022.67	36.68	3946.59	4032.18	37.29	3946.59
R (相关)	4022.67	40.54	3940.08	4032.19	41.11	3940.08

随机种子稳健性: 蒙特卡洛种子 0/1/2 下问题 3 期望收益分别为 4032.19/4032.81/4032.19 万元 (波动 0.6 万元、0.02%), 公共随机数 (CRN) 设计 [16] 下结果与 MC 种子无关。问题 3 在求解器种子 seed=1 (16×40 预算) 下为 4030.48 万元, 与基准 seed=0 全预算下的 4031.87 万元相差 0.03%。DE 与 MC 两类随机源均不改变结论。

6.6 小结

综合以上分析: 数值误差 (MC 抽样、求积) 小于 0.03%; 相关矩阵参数在 $\pm 50\%$ 内扰动对期望收益的影响不超过 0.01%; 波动区间半宽在 $\pm 50\%$ 内缩放的收益变化小于 0.5% (以亩产量影响最大); 分布类型、相关结构误设 ($R = I$)、随机种子均不改变结论。因此本文数值结果可靠, 最优种植方案对建模假设不敏感, 具有较强的稳健性。

七、 模型评价与推广

7.1 模型优点

1. **编码创新**: 全量连续权重编码将 1062 个决策变量全部设为连续实数, 离散变量为零, 消除了组合爆炸, DE 在纯连续空间高效进化。
2. **约束完备**: 六组约束覆盖题目所有要求, 解码器修复链与硬校验双重保障, 所有方案零违规。
3. **计算高效**: 期望收益采用 Gauss-Legendre 解析求积 ($n=16$, 经 16/24/40 三档验证误差小于 0.01 万元), 计算效率远高于蒙特卡洛模拟, 且无随机误差。
4. **方案稳健**: 四种分布假设下期望收益极差不足 1%, 最优方案对分布假设稳健。

7.2 模型缺点

- 问题 3 的相关矩阵 $\mathbf{R}$ 依赖文献设定, 缺乏实测数据验证;
- DE 算法存在随机性, 需多次运行验证稳定性;
- 未考虑气候极端事件等尾部风险因素;
- 模型假设地块面积可分, 实际中可能受机械化作业限制。

7.3 模型推广

当前模型可向以下方向推广: (1) 引入随机规划或鲁棒优化框架, 在最坏情况下保证收益下限; (2) 将相关矩阵 $\mathbf{R}$ 改为时变结构, 刻画市场动态变化; (3) 加入多目标优化 (期望收益 vs 风险), 利用 Pareto 前沿为决策者提供多种选择; (4) 扩展至多乡村联合优化, 考虑区域间的资源调配和市场协调。

参考文献

- [1] 林依硕, 荣梦杰. 基于混合整数线性规划的乡村农作物多情境种植策略优化研究 [J]. 科学发展研究, 2025(5).
- [2] Zhao R, Song Y. Optimized Solution for Crop Planting Strategies Based on Multiple Model Construction[C]. CAMMIC 2025, ACM.
- [3] 许婷婷等. 基于线性规划模型的农作物种植策略 [J]. 佛山科学技术学院学报 (自然科学版), 2025.

- [4] Williams B R, Pounds-Barnett G. Producer Supply Response for Area Planted of Seven Major U.S. Crops[R]. USDA ERS, Economic Research Report No. 327, 2023.
- [5] Kim H, Moschini G. The Dynamics of Supply: U.S. Corn and Soybeans in the Biofuel Era[J]. *Land Economics*, 2018, 94(4): 593–613.
- [6] Mudare S, Jing J, Makowski D, et al. Crop rotations synergize yield, nutrition, and revenue: a meta-analysis[J]. *Nature Communications*, 2025, 16: 9552.
- [7] Yang X, Xiong J, Du T, et al. Diversifying crop rotation increases food production, reduces net greenhouse gas emissions and improves soil health[J]. *Nature Communications*, 2024, 15: 198.
- [8] von Cossel M, Scordia D, Altieri M, et al. Spotlight on agroecological cropping practices to improve the resilience of farming systems[J]. *Frontiers in Agronomy*, 2025, 7: 1495846.
- [9] Storn R, Price K. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces[J]. *Journal of Global Optimization*, 1997, 11(4): 341–359.
- [10] Varian H R. *Microeconomic Analysis*[M]. 3rd ed. New York: Norton, 1992.
- [11] Glasserman P. *Monte Carlo Methods in Financial Engineering*[M]. New York: Springer, 2004.
- [12] Higham N J. Computing the nearest correlation matrix – a problem from finance[J]. *IMA Journal of Numerical Analysis*, 2002, 22(3): 329–343.
- [13] Kleywegt A J, Shapiro A, Homem-de Mello T. The sample average approximation method for stochastic discrete optimization[J]. *SIAM Journal on Optimization*, 2002, 12(2): 479–502.
- [14] Khouja M. The single-period (newsvendor) problem: literature review and suggestions for future research[J]. *European Journal of Operational Research*, 1999, 105(1): 1–13.
- [15] Hicks J R, Allen R G D. A reconsideration of the theory of value[J]. *Economica*, 1934, 1(1/2): 52–76.
- [16] Law A M, Kelton W D. *Simulation Modeling and Analysis*[M]. 3rd ed. New York: McGraw-Hill, 2000.
- [17] 黄凤等. 华北山区农业可持续发展研究 [J]. *农业工程学报*, 2022.

- [18] Cervantes-Gaxiaz D, et al. Agricultural market trends and forecasting: A review[J]. Agricultural Systems, 2020.

附录：完整源程序

以下为求解全部问题的完整 Python 源程序，共 5 个模块。

A.1 data_prep.py（源数据加工）

源程序 1: 源数据加工模块

```
1 # 源数据加工：计算各作物2023总产量(=销售量)。  
2 import pandas as pd  
3  
4 DATA = r'data'  
5  
6 def crop_sales_2023():  
7     # 返回 {作物编号: 2023总产量/斤}。  
8     det = pd.read_csv(f'{DATA}/经济参数明细.csv', skipinitialspace=True)  
9     det['地块类型'] = det['地块类型'].astype(str).strip()  
10    det['种植季次'] = det['种植季次'].astype(str).strip()  
11    det['作物编号'] = pd.to_numeric(det['作物编号']).astype(int)  
12    det['亩产量/斤'] = pd.to_numeric(det['亩产量/斤'])  
13    yield_lookup = {}  
14    for _, r in det.iterrows():  
15        yield_lookup[(r['地块类型'], r['种植季次'], r['作物编号'])] =  
16            r['亩产量/斤']  
17  
18    plots = pd.read_excel('附件1.xlsx', sheet_name='乡村的现有耕地')  
19    plot_type = dict(zip(plots['地块名称'].astype(str).strip(),  
20                        plots['地块类型'].astype(str).strip()))  
21  
22    plant = pd.read_csv(f'{DATA}/2023种植情况.csv', skipinitialspace=True)  
23    plant['种植地块'] = plant['种植地块'].astype(str).strip()  
24    plant['种植季次'] = plant['种植季次'].astype(str).strip()  
25    plant['种植面积/亩'] = pd.to_numeric(plant['种植面积/亩'])  
26  
27    sales = {}  
28    for _, r in plant.iterrows():  
29        j = int(r['作物编号'])  
30        k = (plot_type[r['种植地块']], r['种植季次'], j)  
31        sales[j] = sales.get(j, 0.0) + float(r['种植面积/亩']) * yield_lookup[k]  
32    return {j: round(v, 1) for j, v in sales.items()}
```

A.2 scheme.py（方案表示模块）

源程序 2: 方案表示模块

```
1  # 方案表示：地块0-1选择+面积系数，派生决策变量x供收益核算。
2  from collections import defaultdict
3  import pandas as pd
4
5  DATA = r'data'
6  RICE = 16
7  BEAN = {1, 2, 3, 4, 5, 17, 18, 19}
8  WATER_VEG = set(range(17, 35))
9  WATER_S2 = {35, 36, 37}
10 MUSH = set(range(38, 42))
11
12 class SchemeModel:
13     def __init__(self):
14         self._load()
15
16     def _load(self):
17         d = pd.read_csv(f'{DATA}/地块表.csv', skipinitialspace=True)
18         d['地块编号'] = d['地块编号'].astype(str).strip()
19         self.unit_area, self.unit_support = {}, {}
20         for _, r in d.iterrows():
21             u = r['地块编号']
22             self.unit_area[u] = float(r['面积/亩'])
23             self.unit_support[u] = [int(x) for x in
24                                     str(r['可种植作物编号']).split(';')
25                                     if x.strip().isdigit()]
26         self.plots, self.plot_units = [], defaultdict(list)
27         for u in d['地块编号']:
28             p = u if u[0] in 'ABC' else u[: -2]
29             self.plot_units[p].append(u)
30             if p not in self.plots:
31                 self.plots.append(p)
32         self.plot_type = {p: p[0] for p in self.plots}
33         self.season_support = {}
34         for p in self.plots:
35             if self.plot_type[p] in 'ABC':
36                 self.season_support[p] = {1: self.unit_support[p]}
```

```

36         else:
37             self.season_support[p] = {1: self.unit_support[f'{p}-1'],
38                                     2: self.unit_support[f'{p}-2']}
39
40     def unit_of(self, plot, season):
41         return plot if self.plot_type[plot] in 'ABC' else f'{plot}-{season}'
42
43     def derive(self, plan):
44         x = defaultdict(float)
45         for plot, seasons in plan.items():
46             for s, crops in seasons.items():
47                 u = self.unit_of(plot, s)
48                 for j, alpha in crops:
49                     if alpha > 0:
50                         x[(u, j)] += alpha * self.unit_area[u]
51         return dict(x)

```

A.3 revenue.py（收益模块）

源程序 3: 收益模块（核心片段）

```

1  # 收益模块：问题1确定性+问题2/3期望收益。
2  import re, numpy as np, pandas as pd
3  from data_prep import crop_sales_2023
4
5  DISTS = ('uniform', 'triangular', 'normal', 'logit_normal')
6  GROW_SALES = (6, 7)
7  DATA = r'data'
8
9  def parse_range(s):
10     s = str(s).strip().replace('%', '').replace('+', '')
11     if chr(177) in s:
12         v = float(s.replace(chr(177), ''))
13         return -v, v
14     nums = [float(x) for x in re.findall(r'(?<!\d)-?\d+(?:\.\d+)?', s)]
15     if not nums: return 0.0, 0.0
16     if len(nums) == 1: return nums[0], nums[0]
17     return min(nums), max(nums)
18

```

```

19 def uniform_dist(lo, hi, n=40):
20     if hi <= lo: return np.array([float(lo)]), np.array([1.0])
21     x, w = np.polynomial.legendre.leggauss(n)
22     mid = (lo + hi) / 2
23     x = (hi - lo) / 2 * x + mid
24     return x, w / 2
25
26 def triangular_dist(lo, hi, n=40):
27     if hi <= lo: return np.array([float(lo)]), np.array([1.0])
28     x, w = np.polynomial.legendre.leggauss(n)
29     mid = (lo + hi) / 2
30     x = (hi - lo) / 2 * x + mid
31     w = (hi - lo) / 2 * w
32     f = np.where(x <= mid, 2*(x-lo)/((hi-lo)*(mid-lo)),
33                 2*(hi-x)/((hi-lo)*(hi-mid)))
34     return x, w * f
35
36 def normal_dist(lo, hi, n=40):
37     if hi <= lo: return np.array([float(lo)]), np.array([1.0])
38     x, w = np.polynomial.legendre.leggauss(n)
39     mid = (lo + hi) / 2
40     x = (hi - lo) / 2 * x + mid
41     w = (hi - lo) / 2 * w
42     s = (hi - lo) / 6
43     f = np.exp(-(x-mid)**2/(2*s**2))/(s*np.sqrt(2*np.pi))
44     return x, w * f
45
46 def logit_normal_dist(lo, hi, n=40):
47     if hi <= lo: return np.array([float(lo)]), np.array([1.0])
48     z, w = np.polynomial.hermite.hermgauss(n)
49     x = lo + (hi-lo)/(1+np.exp(-z*np.sqrt(2.0)))
50     return x, w / np.sqrt(np.pi)
51
52 QUAD = {'uniform': uniform_dist, 'triangular': triangular_dist,
53         'normal': normal_dist, 'logit_normal': logit_normal_dist}
54
55 class RevenueModel:
56     def __init__(self):
57         self._load()
58

```

```

59 def _unit_x(self, x):
60     if isinstance(x, np.ndarray): return np.asarray(x, dtype=float)
61     X = np.zeros((len(self.units), len(self.crops)))
62     for (u, j), area in x.items():
63         if area > 0:
64             iu = u if isinstance(u, int) else self._unit_idx[u]
65             X[iu, self.crop_idx[j]] += area
66     return X
67
68 def to_ts(self, x):
69     X_unit = self._unit_x(x)
70     X_ts = np.zeros((len(self.ts_list), len(self.crops)))
71     for it in range(len(self.ts_list)):
72         X_ts[it] = (X_unit * (self.ts_map == it)).sum(axis=0)
73     return X_ts
74
75 def _load(self):
76     self.ts_list = ['A', 'B', 'C', 'D', 'D-1', 'D-2', 'E-1', 'E-2', 'F-1', 'F-2']
77     self.ts_idx = {t: i for i, t in enumerate(self.ts_list)}
78     self.crops = list(range(1, 42))
79     self.crop_idx = {j: j-1 for j in self.crops}
80     d = pd.read_csv(f'{DATA}/经济参数明细.csv', skipinitialspace=True)
81     Tq = np.full((len(self.ts_list), len(self.crops)), np.nan)
82     Tc = np.full_like(Tq, np.nan); Tp = np.full_like(Tq, np.nan)
83     for _, r in d.iterrows():
84         it = self.ts_idx[str(r['类型-季节编号']).strip()]
85         jc = self.crop_idx[int(r['作物编号'])]
86         Tq[it, jc], Tc[it, jc], Tp[it, jc] = (float(r['亩产量/斤']),
87             float(r['种植成本/(元/亩)']), float(r['售价中值/(元/斤)']))
88     self.q = np.nan_to_num(Tq); self.c = np.nan_to_num(Tc)
89     self.p = np.nan_to_num(Tp)
90     plot = pd.read_csv(f'{DATA}/地块表.csv', skipinitialspace=True)
91     plot['地块编号'] = plot['地块编号'].astype(str).str.strip()
92     self.units = list(plot['地块编号'])
93     self._unit_idx = {u: i for i, u in enumerate(self.units)}
94     self.unit_area = np.array(pd.to_numeric(plot['面积/亩']), dtype=float)
95     self.support = {}
96     for i, r in plot.iterrows():
97         self.support[i] = {self.crop_idx[int(x)] for x in
            str(r['可种植作物编号']).split(';')}

```

```

98         if x.strip().isdigit()}
99     self.ts_map = np.full((len(self.units), len(self.crops)), -1, dtype=int)
100     for iu in range(len(self.units)):
101         for jc in self.support[iu]:
102             self.ts_map[iu, jc] = self.ts_idx[self._ts_of(iu, jc)]
103     sales0 = crop_sales_2023()
104     self.sales0 = np.array([sales0[j] for j in self.crops])
105     s = pd.read_csv(f'{DATA}/经济数据.csv', skipinitialspace=True)
106     s['作物编号'] = pd.to_numeric(s['作物编号'])
107     self.wave_sales = np.array([parse_range(s.loc[s['作物编号']==j,
108         '预期销售量年变化率'].iloc[0]) for j in self.crops])
109     self.wave_yield = np.array([parse_range(s.loc[s['作物编号']==j,
110         '亩产量年变化率'].iloc[0]) for j in self.crops])
111     self.wave_price = np.array([parse_range(s.loc[s['作物编号']==j,
112         '销售价格年变化率'].iloc[0]) for j in self.crops])
113
114     def _ts_of(self, iu, jc):
115         u = self.units[iu]; head = u[0]
116         if head in 'ABC': return head
117         if head == 'D':
118             if u.endswith('-2'): return 'D-2'
119             return 'D' if jc == 15 else 'D-1'
120         if head == 'E': return 'E-1' if u.endswith('-1') else 'E-2'
121         return 'F-1' if u.endswith('-1') else 'F-2'
122
123     def profit_det(self, x, mode=1):
124         X = self.to_ts(x)
125         prod = (self.q * X).sum(axis=0)
126         gross = (self.p * self.q * X).sum(axis=0)
127         cost = float((self.c * X).sum())
128         with np.errstate(divide='ignore', invalid='ignore'):
129             r = np.where(prod > 0, self.sales0 / prod, 1.0)
130             r = np.minimum(r, 1.0)
131             kappa = 0.5 if mode == 2 else 0.0
132             rev = float((gross * (r + kappa*(1.0-r))).sum())
133             return rev - cost
134
135     def profit_stoch(self, x, year, dist='normal', mode=1, n_quad=40):
136         u = year - 2023; X = self.to_ts(x)
137         Yb = (self.q*X).sum(axis=0); PX = (self.p*self.q*X).sum(axis=0)

```

```

138 C = float((self.c*X).sum()*(1.05**u)
139 with np.errstate(divide='ignore', invalid='ignore'):
140     A = np.where(Yb>0, self.sales0/Yb, 0.0)
141 grow = np.zeros(len(self.crops), dtype=bool)
142 grow[list(self.crop_idx[j] for j in GROW_SALES)] = True
143 kappa = 0.5 if mode==2 else 0.0
144 plo, phi = self.wave_price[:,0], self.wave_price[:,1]
145 E = 0.0
146 for jc in range(len(self.crops)):
147     if A[jc]<=0 or PX[jc]<=0: continue
148     pl, ph = plo[jc], phi[jc]
149     if pl==ph:
150         fp = 1.0 if pl==0 else (1+pl/100.0)**u
151     else:
152         xp, wp = self._quad(pl/100.0, ph/100.0, dist, n_quad)
153         fp = float(((1+xp)**u*wp).sum())
154     xq, wq = self._quad(self.wave_yield[jc,0]/100.0,
155                         self.wave_yield[jc,1]/100.0, dist, n_quad)
156     xs, ws = self._quad(self.wave_sales[jc,0]/100.0,
157                         self.wave_sales[jc,1]/100.0, dist, n_quad)
158     gj = (lambda v: (1+v)**u) if grow[jc] else (lambda v: 1+v)
159     den = 1.0+xq; gb = gj(xs)
160     r = np.minimum(1.0, A[jc]*gb[None,:]/den[:,None])
161     psi = r+kappa*(1.0-r)
162     I = float((den[:,None]*psi*wq[:,None]*ws[None,:]).sum())
163     E += PX[jc]*fp*I
164 E -= C; return E
165
166 def corr_matrix(self):
167     if getattr(self, '_R', None) is not None: return self._R
168     m = len(self.crops); R = np.eye(3*m)
169     for j in range(m):
170         d,p,c = 3*j, 3*j+1, 3*j+2
171         R[d,p]=R[p,d]=0.40; R[d,c]=R[c,d]=0.30; R[p,c]=R[c,p]=0.20
172     for a,b,rho in self.COMP_PAIRS:
173         R[3*(a-1),3*(b-1)]=R[3*(b-1),3*(a-1)]=rho
174     for a,b,mag in self._sub_pairs():
175         R[3*(a-1),3*(b-1)]=R[3*(b-1),3*(a-1)]=-mag
176     w,V = np.linalg.eigh(R); w = np.clip(w, 1e-8, None)
177     R2 = (V*w)@V.T; dg = np.sqrt(np.diag(R2))

```

```

178     self._R = R2/np.outer(dg,dg); return self._R
179
180 def mc_samples(self, year, N=2000, R=None, seed=0):
181     rng = np.random.default_rng(seed); m = len(self.crops)
182     dlo,dhi = self.wave_sales[:,0]/100.0, self.wave_sales[:,1]/100.0
183     plo,phi = self.price_range_p3()
184     clo = np.full(m,-0.02); chi = np.full(m,0.02)
185     lo = np.concatenate([dlo,plo,clo]); hi = np.concatenate([dhi,phi,chi])
186     mu = 0.5*(lo+hi); sig = (hi-lo)/6.0
187     z0 = rng.standard_normal((N,3*m))
188     if R is None: z = mu+sig*z0
189     else:
190         S = R*sig[:,None]*sig[None,:]; w,V = np.linalg.eigh(S)
191         w = np.clip(w,1e-10,None); L = V*np.sqrt(w); z = mu+z0@L.T
192     z = np.clip(z,lo,hi)
193     xiq = np.clip(0.20/6.0*rng.standard_normal((N,m)),-0.10,0.10)
194     return {'xiq':xiq,'xid':z[:,m],'xip':z[:,m:2*m],'xic':z[:,2*m:]}
195
196 def profit_mc(self, x, year, mode=1, N=2000, R=None, seed=0, samples=None):
197     u = year-2023; X = self.to_ts(x)
198     Yb = (self.q*X).sum(axis=0); PX = (self.p*self.q*X).sum(axis=0)
199     Cj = (self.c*X).sum(axis=0)
200     with np.errstate(divide='ignore', invalid='ignore'):
201         A = np.where(Yb>0, self.sales0/Yb, 0.0)
202     kappa = 0.5 if mode==2 else 0.0
203     grow = np.zeros(len(self.crops), dtype=bool)
204     grow[list(self.crop_idx[j] for j in GROW_SALES)] = True
205     if samples is None: samples = self.mc_samples(year,N,R,seed)
206     xiq,xid,xip,xic =
207         (samples['xiq'],samples['xid'],samples['xip'],samples['xic'])
208     fq = 1.0+xiq
209     fd = np.where(grow[None,:], (1.0+xid)**u, 1.0+xid)
210     fp = (1.0+xip)**u; Dt = self.sales0[None,:]*fd; prod = Yb[None,:]*fq
211     with np.errstate(divide='ignore', invalid='ignore'):
212         r = np.minimum(1.0, np.where(prod>0, Dt/prod, 1.0))
213     psi = r+kappa*(1.0-r); gross = PX[None,:]*fp*fq
214     rev = (gross*psi).sum(axis=1)
215     cost = (Cj[None,:]*(1.0+xic)*(1.05**u)).sum(axis=1)
216     return rev-cost

```


A.4 rule_checker.py（规则检查器）

源程序 4: 规则检查器模块

```
1  # 规则检查器+方案整合器。
2  from collections import defaultdict, Counter
3  from dataclasses import dataclass, field
4  from scheme import BEAN, MUSH, RICE, WATER_S2, WATER_VEG, SchemeModel
5
6  YEARS = range(2024, 2031)
7
8  @dataclass
9  class Violation:
10     rule: str; year: int; plot: str; season: int; detail: str
11     def __str__(self):
12         s = f'{self.year}' if self.year else ' '
13         return f'[{self.rule}] {s}年 {self.plot} 季{self.season}: {self.detail}'
14
15  @dataclass
16  class Report:
17     violations: list = field(default_factory=list)
18     @property
19     def ok(self): return not self.violations
20     def summary(self): return dict(Counter(v.rule for v in self.violations))
21
22  class RuleChecker:
23     def __init__(self, scheme=None):
24         self.scheme = scheme or SchemeModel()
25
26     def crop_sets(self, plan):
27         return {p: {s: {j for j, a in crops if a > 0}
28                     for s, crops in seasons.items()}}
29             for p, seasons in plan.items()
30
31     def replant_violations(self, prev, curr):
32         out = []
33         for p in set(prev) | set(curr):
34             seq = []
35             for d in (prev, curr):
36                 for s in (1, 2):
```

```

37         cs_ = d.get(p, {}).get(s)
38         if cs_: seq.append(cs_)
39     for i in range(len(seq)-1):
40         inter = seq[i] & seq[i+1]
41         if inter: out.append((p, f'相邻两茬重叠 {inter}'))
42     return out
43
44 def bean_violations(self, years):
45     sets = {y: self.crop_sets(p) for y, p in years.items()}
46     out = []
47     for p in self.scheme.plots:
48         for w in range(2023, 2029):
49             hit = any(seasons & BEAN
50                       for y in (w, w+1, w+2)
51                             for seasons in sets.get(y, {}).get(p, {}).values())
52             if not hit: out.append((p, w))
53     return out
54
55 def penalty(self, years, lam1=1e6, lam2=1e6):
56     n1 = sum(len(self.replant_violations(self.crop_sets(years[t-1]),
57                                           self.crop_sets(years[t])))
58             for t in range(2024, 2031) if t in years and t-1 in years)
59     n2 = len(self.bean_violations(years))
60     return lam1*n1 + lam2*n2
61
62 def check_year(self, plan, year=0):
63     vs = []
64     for plot, seasons in plan.items():
65         t = self.scheme.plot_type.get(plot)
66         if t is None:
67             vs.append(Violation('S1', year, plot, 0, '未知地块')); continue
68         for s, crops in seasons.items():
69             if s not in self.scheme.season_support[plot]:
70                 vs.append(Violation('S2', year, plot, s, '该季次不存在'));
71                 continue
72             sup = set(self.scheme.season_support[plot][s])
73             seen, sa = {}, 0.0
74             for j, a in crops:
75                 sa += a
76                 if not (0.0<=a<=1.0+1e-9):

```

```

76         vs.append(Violation('S4', year, plot, s, f'alpha={a:.3f}
77             越界'))
78     if a>0 and j not in sup:
79         vs.append(Violation('S3', year, plot, s, f'作物{j}
80             不在支持集'))
81     if a>0: seen[j] = seen.get(j,0)+1
82     if sa>1+1e-9:
83         vs.append(Violation('S4', year, plot, s, f'Sigma={sa:.3f} >
84             1'))
85     for j, n in seen.items():
86         if n>1: vs.append(Violation('S5', year, plot, s, f'作物{j}
87             重复 {n} 条'))
88 if t=='D':
89     s1 = {j for j,a in seasons.get(1,[]) if a>0}
90     s2 = {j for j,a in seasons.get(2,[]) if a>0}
91     if s1&WATER_S2:
92         vs.append(Violation('W4',year,plot,1,f'第一季含第二季作物'))
93     if RICE in s1 and s1-{RICE}:
94         vs.append(Violation('W1',year,plot,1,'水稻与蔬菜互斥'))
95     if RICE in s1 and s2:
96         vs.append(Violation('W1',year,plot,2,'单季水稻第二季应为空'))
97     if s2 and not (s1&WATER_VEG):
98         vs.append(Violation('W5',year,plot,2,'第二季必须依附双季蔬菜'))
99     if s1&WATER_VEG and not s2:
100         vs.append(Violation('W2',year,plot,2,'双季模式第二季应选恰好一种'))
101     if s1&WATER_VEG and s2 and len(s2)>1:
102         vs.append(Violation('W3',year,plot,2,'双季模式第二季至多一种'))
103 if t=='E':
104     s1 = {j for j,a in seasons.get(1,[]) if a>0}
105     s2 = {j for j,a in seasons.get(2,[]) if a>0}
106     if s1&WATER_S2 or s1&MUSH:
107         vs.append(Violation('P1',year,plot,1,'普通大棚一季应为蔬菜'))
108     if s2 and not s2<=MUSH:
109         vs.append(Violation('P1',year,plot,2,'普通大棚二季只能食用菌'))
110 if t=='F':
111     for s,crops in seasons.items():
112         js = {j for j,a in crops if a>0}
113         if js&(WATER_S2|MUSH):
114             vs.append(Violation('P2',year,plot,s,'智慧大棚只能蔬菜'))
115 return vs

```

```

112
113 def check_full(self, full, base=None):
114     full = dict(full)
115     if base: full = {2023: base, **full}
116     vs = []
117     for y, plan in full.items():
118         vs += self.check_year(plan, year=y)
119         missing = set(self.scheme.plots)-set(plan)
120         if missing:
121             vs.append(Violation('F1',y,str(sorted(missing)[:6]),0,
122                               f'缺 {len(missing)} 块地'))
123     years = sorted(y for y in full if y in YEARS)
124     for i, y in enumerate(years):
125         prev = full[y-1] if y-1 in full else None
126         if prev is None: continue
127         for p, msg in self.replant_violations(self.crop_sets(prev),
128                                              self.crop_sets(full[y])):
129             vs.append(Violation('R1',y,p,0,msg))
130     for p, w in self.bean_violations(full):
131         vs.append(Violation('B1',w,p,0,f'窗口{w}~{w+2}无豆类'))
132     return Report(vs)
133
134 def raise_if_invalid(self, full, base=None):
135     rep = self.check_full(full, base=base)
136     if not rep.ok:
137         msg = f'方案违反{len(rep.violations)}条规则: \n'+'\n'.join(f' {v}'
138                               for v in rep.violations)
139         raise ValueError(msg)
140     return rep
141
142 def integrate(plans, base=None, scheme=None):
143     scheme = scheme or SchemeModel(); out = {}
144     for y, plan in plans.items():
145         nyr = {}
146         for p in scheme.plots:
147             ss = {}
148             for s, crops in plan.get(p,{}).items():
149                 agg = defaultdict(float)
150                 for j,a in crops:
151                     if a>0: agg[j] += a

```

```

151         if agg: ss[s] = sorted((j,a) for j,a in agg.items())
152         nyr[p] = ss
153         out[int(y)] = nyr
154     if base: out[2023] = base
155     return out

```

A.5 de_solver_full.py（差分进化求解器）

源程序 5: 差分进化求解器（核心片段）

```

1  # 全量连续权重编码+差分进化求解器。
2  import pickle, random
3  from collections import defaultdict
4  from pathlib import Path
5  import numpy as np, pandas as pd
6  from revenue import RevenueModel
7  from rule_checker import RuleChecker, integrate
8  from scheme import BEAN, RICE, WATER_S2, WATER_VEG, SchemeModel
9
10 YEARS = range(2024, 2031)
11 VEG = list(range(17, 35))
12 MUSH = list(range(38, 41))
13 S2 = sorted(WATER_S2)
14 NP=32; G=80; F_=0.7; CR=0.9; LAM=1e6; SPARSE=50.0; EPS=0.005
15
16 class FullDE:
17     def __init__(self, scheme=None, revenue=None, seed=0):
18         self.seed = seed
19         self.sm = scheme or SchemeModel()
20         self.rev = revenue or RevenueModel()
21         self.rc = RuleChecker(self.sm)
22         self.rng = random.Random(seed)
23         self.unit_idx = {u: i for i, u in enumerate(self.rev.units)}
24         self.crop_idx = {j: j-1 for j in range(1, 42)}
25         self.margin = self._margin_table()
26         self.blocks, off = [], 0
27         for p in self.sm.plots:
28             for s in sorted(self.sm.season_support[p]):
29                 sup = list(self.sm.season_support[p][s])

```

```

30         self.blocks.append((p, s, sup, off))
31         off += len(sup)
32     self.D = off # 1062
33     self.mc_N=2000; self.mc_seed=0; self.mc_R=None; self.mc_cache={}
34
35     def _margin_table(self):
36         M = np.full((len(self.rev.units), 41), -np.inf)
37         for iu in range(len(self.rev.units)):
38             for jc in range(41):
39                 ts = self.rev.ts_map[iu, jc]
40                 if ts >= 0:
41                     M[iu,jc] =
42                         self.rev.p[ts,jc]*self.rev.q[ts,jc]-self.rev.c[ts,jc]
43
44         return M
45
46     def _mg(self, plot, s, j):
47         if j is None: return -np.inf
48         u = self.sm.unit_of(plot, s)
49         return float(self.margin[self.unit_idx[u], self.crop_idx[j]])
50
51     def _fill(self, p, s, cand, demand, mode=1, exclude=(),
52               single=False, max_crops=4):
53         u = self.sm.unit_of(p, s); A_u = float(self.sm.unit_area[u])
54         kappa = 0.5 if mode==2 else 0.0
55         out, tot, remain = [], 0.0, 1.0; taken = set(exclude)
56         while remain>1e-6 and len(out)<max_crops:
57             best = None
58             for j in cand:
59                 if j in taken: continue
60                 jc = self.crop_idx[j]
61                 ts = self.rev.ts_map[self.unit_idx[u], jc]
62                 if ts < 0: continue
63                 q = float(self.rev.q[ts,jc]); pj = float(self.rev.p[ts,jc])
64                 c = float(self.rev.c[ts,jc])
65                 if q<=0 or pj<=0: continue
66                 D = float(self.rev.sales0[jc]); rem = demand[j]; Y_u = q*A_u
67                 if rem > 1e-6:
68                     a = min(remain, rem/Y_u); r = 1.0
69                 else:
68                     a = remain; r = 0.0

```

```

69         if a <= 1e-6: continue
70         psi = r+kappa*(1.0-r); m = psi*pj*Y_u*a-c*A_u*a
71         if m <= 0: continue
72         if best is None or m > best[0]: best = (m, j, a)
73     if best is None: break
74     m, j, a = best; out.append((j,a)); tot += m
75     jc = self.crop_idx[j]; ts = self.rev.ts_map[self.unit_idx[u], jc]
76     demand[j] -= a*float(self.rev.q[ts,jc])*A_u; remain -= a
77     if single: break
78     taken.add(j)
79     return out, tot
80
81 def _greedy_year(self, prev_sets, mode=1):
82     demand = {j: float(self.rev.sales0[j-1]) for j in range(1, 42)}
83     order = sorted(self.sm.plots,
84         key=lambda p: -max((self._mg(p,s,j)
85             for s in self.sm.season_support[p]
86             for j in self.sm.season_support[p][s]), default=-np.inf))
87     plan = {}
88     for p in order:
89         t = self.sm.plot_type[p]; seasons = {}
90         if t in 'ABC':
91             cs_, _ = self._fill(p,1,self.sm.season_support[p][1],
92                 demand,mode,exclude=prev_sets.get(p,{}).get(1,set()))
93             if cs_: seasons[1] = cs_
94         elif t == 'D':
95             ex1 =
96                 prev_sets.get(p,{}).get(1,set())|prev_sets.get(p,{}).get(2,set())
97             d_r,s_r,m_r = dict(demand),{},0.0
98             c1,m1 = self._fill(p,1,[RICE],d_r,mode,exclude=ex1)
99             if c1: s_r[1],m_r = c1,m1
100             d_v,s_v = dict(demand),{}
101             c1,m1 = self._fill(p,1,VEG,d_v,mode,exclude=ex1)
102             c2,m2 = self._fill(p,2,S2,d_v,mode,single=True,
103                 exclude=prev_sets.get(p,{}).get(2,set()))
104             if c1:
105                 s_v[1] = c1
106                 if c2: s_v[2] = c2
107             m_v = m1+m2 if s_v.get(1) else -np.inf
108             if m_v>m_r and s_v.get(1): seasons,demand = s_v,d_v

```

```

108         elif s_r: seasons,demand = s_r,d_r
109     elif t == 'E':
110         c1,m1 = self._fill(p,1,VEG,demand,mode)
111         c2,m2 = self._fill(p,2,MUSH+[41],demand,mode)
112         if c1: seasons[1] = c1
113         if c2: seasons[2] = c2
114     else:
115         ex1 = prev_sets.get(p,{}).get(2,set())
116         c1,m1 = self._fill(p,1,VEG,demand,mode,exclude=ex1)
117         ex2 = {j for j,_ in c1}
118         c2,m2 = self._fill(p,2,VEG,demand,mode,exclude=ex2)
119         if c1: seasons[1] = c1
120         if c2: seasons[2] = c2
121     plan[p] = seasons
122     return plan
123
124 def _encode(self, plan_y):
125     vec = np.zeros(self.D)
126     for p,s,sup,off in self.blocks:
127         for j,a in plan_y.get(p,{}).get(s,[]):
128             if j in sup: vec[off+sup.index(j)] = a
129     return vec
130
131 def _decode(self, vec, y, prev_sets):
132     plan = defaultdict(lambda: defaultdict(list))
133     for p,s,sup,off in self.blocks:
134         t = self.sm.plot_type[p]; adj = set()
135         if s==1:
136             ps2 = set(prev_sets.get(p,{}).get(2,[]))
137             ps1 = set(prev_sets.get(p,{}).get(1,[]))
138             adj = ps2|(ps1 if not ps2 else set())
139         else:
140             adj = {j for j,_ in plan[p].get(1,[])}
141         w = {j: max(0.0,vec[off+i]) for i,j in enumerate(sup)}
142         if t=='D' and s==1:
143             if RICE not in adj and w.get(RICE,0.0)>0.02:
144                 plan[p][1].append((RICE,1.0)); continue
145             w.pop(RICE,None)
146         if adj:
147             for j in adj: w.pop(j,None)

```



```

148         total = sum(w.values())
149         if total > 1.0:
150             k = 1.0/total; w = {j:a*k for j,a in w.items()}
151         for j,a in w.items():
152             if a >= EPS: plan[p][s].append((j,a))
153     # D地块模式修复
154     for p,seasons in list(plan.items()):
155         if self.sm.plot_type[p]!='D': continue
156         s1 = {j for j,a in seasons.get(1,[]) if a>0}
157         s2 = {j for j,a in seasons.get(2,[]) if a>0}
158         if RICE in s1 and s1-{RICE}:
159             seasons[1] = [(RICE,next(a for j,a in seasons[1] if j==RICE))]
160             s1={RICE}
161         if RICE in s1 and s2: del seasons[2]; s2=set()
162         if s2 and not (s1&WATER_VEG): del seasons[2]; s2=set()
163         if (s1&WATER_VEG) and not s2:
164             adj = set(s1); ok = [j for j in WATER_S2 if j not in adj]
165             if ok: seasons[2] = [(max(ok,key=lambda j:self._mg(p,2,j)),1.0)]
166     # E/F塌缩修复
167     for p,seasons in list(plan.items()):
168         if self.sm.plot_type[p] not in ('E','F'): continue
169         s2 = seasons.get(2,[])
170         if s2 and not seasons.get(1):
171             ps2 = set(prev_sets.get(p,{}).get(2,[]))
172             ps1 = set(prev_sets.get(p,{}).get(1,[]))
173             sup = list(self.sm.season_support[p][1])
174             adj = set(ps2)|{j for j,_ in s2}|(ps1 if not ps2 else set())
175             ok = [j for j in sup if j not in adj]
176             if ok: seasons[1] = [(max(ok,key=lambda j:self._mg(p,1,j)),1.0)]
177             else: del seasons[2]
178     out = {p: dict(s) for p,s in plan.items()}
179     for p in self.sm.plots: out.setdefault(p,{})
180     return out
181
182     def _repair_beans(self, plan):
183         for p in self.sm.plots:
184             for w in range(2023, 2029):
185                 def beans(y):
186                     return {j for s,cs_ in plan.get(y,{}).get(p,{}).items()
187                             for j,a in cs_ if a>0 and j in BEAN}

```

```

188         if any(beans(y) for y in (w,w+1,w+2)): continue
189         cand = []
190         for y in (w,w+1,w+2):
191             if y==2023 or y not in plan or p not in plan[y]: continue
192             for s,cs_ in plan[y][p].items():
193                 sup = self.sm.season_support[p][s]
194                 if not (set(sup)&BEAN): continue
195                 j = cs_[0][0] if cs_ else None
196                 cand.append((self._mg(p,s,j) if j else 0.0, y, s))
197         if not cand: continue
198         _,y,s = min(cand)
199         adj = self._adjacent(plan,p,y,s)
200         ok = [j for j in self.sm.season_support[p][s] if j in BEAN and j
201                not in adj]
202         if not ok: continue
203         nb = max(ok, key=lambda j: self._mg(p,s,j))
204         if self.sm.plot_type[p]=='D' and s==1 and not plan[y][p].get(2):
205             adj2 = self._adjacent(plan,p,y,2)|{nb}
206             ok2 = [j for j in S2 if j not in adj2]
207             if not ok2: continue
208             plan[y][p][2] = [(max(ok2,key=lambda j:self._mg(p,2,j)),1.0)]
209             plan[y][p][s] = [(nb,1.0)]
210
211     def _adjacent(self, plan, p, y, s):
212         adj = set()
213         if s==1:
214             for y2 in (y-1,y):
215                 for jj,_ in plan.get(y2,{}).get(p,{}).get(2,[]): adj.add(jj)
216             for y2 in (y-1,y+1):
217                 for jj,_ in plan.get(y2,{}).get(p,{}).get(1,[]): adj.add(jj)
218         else:
219             for y2 in (y,y+1):
220                 for jj,_ in plan.get(y2,{}).get(p,{}).get(1,[]): adj.add(jj)
221             for y2 in (y-1,y+1):
222                 for jj,_ in plan.get(y2,{}).get(p,{}).get(2,[]): adj.add(jj)
223         return adj
224
225     def _bean_in(self, crop_sets_p):
226         return any(s&BEAN for s in crop_sets_p.values())

```

```

227 def _bean_close(self, plan_y, y, prev_sets, prev2_sets):
228     if y < 2025: return 0
229     n = 0
230     for p in self.sm.plots:
231         if self._bean_in(prev2_sets.get(p, {})) or
232            self._bean_in(prev_sets.get(p, {})):
233             continue
234         if not
235             self._bean_in(self.rc.crop_sets({p: plan_y.get(p, {})}).get(p, {})):
236             n += 1
237     return n
238
239 def _n_crops(self, plan_y):
240     return sum(1 for _, seas in plan_y.items()
241                for _, cs_ in seas.items() for _, a in cs_ if a > 0)
242
243 def _virtual_penalty(self, plan_y, y, prev_sets, prev2_sets):
244     n_enc = len(self.rc.check_year(plan_y, y))
245     n_re =
246         len(self.rc.replant_violations(prev_sets, self.rc.crop_sets(plan_y)))
247     n_bean = self._bean_close(plan_y, y, prev_sets, prev2_sets)
248     return LAM * (n_enc + n_re + n_bean) + SPARSE * self._n_crops(plan_y)
249
250 def _fitness(self, plan_y, y, prev_sets, prev2_sets, problem, mode, dist,
251              n_quad):
252     x = self.sm.derive(plan_y)
253     if problem == 1: profit = self.rev.profit_det(x, mode)
254     elif problem == 3:
255         profit = float(self.rev.profit_mc(x, y, mode, N=self.mc_N, R=self.mc_R,
256                                           samples=self._mc_samples(y)).mean())
257     else: profit = self.rev.profit_stoch(x, y, dist, mode, n_quad)
258     return profit - self._virtual_penalty(plan_y, y, prev_sets, prev2_sets)
259
260 def _mc_samples(self, y):
261     if y not in self.mc_cache:
262         self.mc_cache[y] =
263             self.rev.mc_samples(y, N=self.mc_N, R=self.mc_R, seed=self.mc_seed)
264     return self.mc_cache[y]
265
266 def _need_bean_plots(self, y, prev_sets, prev2_sets):

```

```

262     if y<2025: return []
263     return [p for p in self.sm.plots
264             if not self._bean_in(prev2_sets.get(p,{}))
265             and not self._bean_in(prev_sets.get(p,{}))]
266
267     def _force_bean(self,vec,p,prev_sets):
268         for pp,s,sup,off in self.blocks:
269             if pp!=p or s!=1: continue
270             if not any(j in BEAN for j in sup): return False
271             ps2 = set(prev_sets.get(p,{}).get(2,[]))
272             ps1 = set(prev_sets.get(p,{}).get(1,[]))
273             adj = ps2|(ps1 if not ps2 else set())
274             ok = [j for j in sup if j in BEAN and j not in adj]
275             if not ok: return False
276             j = max(ok,key=lambda j:self._mg(p,1,j))
277             vec[off+sup.index(j)] = 1.0
278             if self.sm.plot_type[p]=='D':
279                 rice_i = sup.index(RICE) if RICE in sup else -1
280                 if rice_i>=0: vec[off+rice_i] = 0.0
281             return True
282         return False
283
284     def _de_year(self, vec0, y, prev_sets, prev2_sets, problem, mode, dist,
285                 n_quad,
286                 npop=NP, ngen=G, F_=F_, CR_=CR):
287         rng = np.random.default_rng(self.seed*1000+y); D = self.D
288         need_bean = self._need_bean_plots(y,prev_sets,prev2_sets)
289         pop = np.zeros((npop,D)); pop[0] = vec0
290         for i in range(1,npop):
291             v = vec0.copy(); nz = np.nonzero(vec0)[0]
292             if len(nz): v[nz] =
293                 np.clip(v[nz]+rng.uniform(-0.15,0.15,len(nz)),0.0,1.0)
294             for _ in range(int(rng.integers(1,4))):
295                 b = int(rng.integers(len(self.blocks)))
296                 _,_,sup,off = self.blocks[b]
297                 v[off+int(rng.integers(len(sup)))]= rng.uniform(0.2,1.0)
298                 pop[i] = np.clip(v,0.0,1.0)
299             for i in range(npop):
300                 for p in need_bean: self._force_bean(pop[i],p,prev_sets)
301             fits = [self._fitness(self._decode(pop[i],y,prev_sets),

```

```

300         y,prev_sets,prev2_sets,problem,mode,dist,n_quad) for i in
                range(npop)]
301     for _ in range(nngen):
302         for i in range(npop):
303             a,b,c = rng.choice([j for j in range(npop) if
                j!=i],3,replace=False)
304             v = pop[a]+F_*(pop[b]-pop[c]); trial = pop[i].copy()
305             mask = rng.random(D)<CR_; mask[rng.integers(D)] = True
306             trial[mask] = v[mask]; trial = np.clip(trial,0.0,1.0)
307             ft = self._fitness(self._decode(trial,y,prev_sets),
308                 y,prev_sets,prev2_sets,problem,mode,dist,n_quad)
309             if ft>fits[i]: pop[i],fits[i] = trial,ft
310         bi = int(np.argmax(fits)); return pop[bi],fits[bi]
311
312     def _polish(self,plan,problem,mode,dist,n_quad,max_iter=6):
313         for _ in range(max_iter):
314             changed = False
315             profs = {yy:self._profit(plan,yy,problem,mode,dist,n_quad) for yy
                in YEARS}
316             for y in YEARS:
317                 prev_sets = self.rc.crop_sets(plan[y-1])
318                 demand = self._demand_state(plan[y])
319                 other = sum(v for k,v in profs.items() if k!=y)
320                 for p in self.sm.plots:
321                     cur = plan[y][p]
322                     base = other+profs[y]-self.rc.penalty(plan)
323                     best_trial, best_f = cur, base
324                     for trial in
                        self._candidates(p,prev_sets.get(p,{}),demand,mode):
325                         if trial==cur: continue
326                         plan[y][p] = trial
327                         fy = self._profit(plan,y,problem,mode,dist,n_quad)
328                         f = other+fy-self.rc.penalty(plan)
329                         if f>best_f: best_f,best_trial = f,trial
330                         plan[y][p] = cur
331                     if best_trial is not cur:
332                         plan[y][p] = best_trial
333                         profs[y] = self._profit(plan,y,problem,mode,dist,n_quad)
334                         changed = True
335             if not changed: break

```

```

336
337 def _demand_state(self, plan_y):
338     used = {j:0.0 for j in range(1,42)}
339     for p, seasons in plan_y.items():
340         for s, cs_ in seasons.items():
341             for j, a in cs_:
342                 u = self.sm.unit_of(p, s); jc = self.crop_idx[j]
343                 ts = self.rev.ts_map[self.unit_idx[u], jc]
344                 used[j] +=
345                     a*float(self.sm.unit_area[u])*float(self.rev.q[ts, jc])
346
347     return {j:float(self.rev.sales0[j-1])-used[j] for j in range(1,42)}
348
349 def _candidates(self, p, prev_sets, demand, mode=1):
350     t = self.sm.plot_type[p]
351     def top(s, cand, n=3, exclude=()):
352         return sorted([j for j in cand if j not in exclude],
353             key=lambda j: -self._eff_margin(p, s, j, demand[j], mode))[:n]
354     outs = [{}]
```

if t in 'ABC':

```

355         for j in
356             top(1, self.sm.season_support[p][1], exclude=prev_sets.get(1, set())):
357             outs.append({1: [(j, 1.0)]})
358     elif t == 'D':
359         outs.append({1: [(RICE, 1.0)]})
360         for s1v in top(1, VEG, 3):
361             for s2v in top(2, S2, 2):
362                 outs.append({1: [(s1v, 1.0)], 2: [(s2v, 1.0)]})
363     elif t == 'E':
364         for s1 in top(1, VEG, 3):
365             for s2 in top(2, MUSH+[41], 3):
366                 outs.append({1: [(s1, 1.0)], 2: [(s2, 1.0)]})
367     else:
368         ex = prev_sets.get(2, set())
369         for s1 in top(1, VEG, 3, exclude=ex):
370             for s2 in top(2, VEG, 3, exclude={s1}):
371                 outs.append({1: [(s1, 1.0)], 2: [(s2, 1.0)]})
372     return outs
373
374 def _eff_margin(self, plot, s, j, rem, mode=1):
375     u = self.sm.unit_of(plot, s); jc = self.crop_idx[j]

```

```

374     ts = self.rev.ts_map[self.unit_idx[u],jc]
375     if ts<0: return -np.inf
376     q=float(self.rev.q[ts,jc]); p=float(self.rev.p[ts,jc])
377     c=float(self.rev.c[ts,jc]); A_u=float(self.sm.unit_area[u])
378     D=float(self.rev.sales0[jc])
379     Y_new = (D-max(rem,0.0))+q*A_u
380     r = min(1.0,D/Y_new) if Y_new>0 else 1.0
381     kappa = 0.5 if mode==2 else 0.0
382     return (r+kappa*(1.0-r))*p*q*A_u-c*A_u
383
384 def _profit(self,plan,y,problem,mode,dist,n_quad):
385     x = self.sm.derive(plan[y])
386     if problem==1: return self.rev.profit_det(x,mode)
387     if problem==3:
388         return float(self.rev.profit_mc(x,y,mode,N=self.mc_N,R=self.mc_R,
389             samples=self._mc_samples(y)).mean())
390     return self.rev.profit_stoch(x,y,dist=dist,mode=mode,n_quad=n_quad)
391
392 def fitness(self,plan,problem=1,mode=1,dist='normal',n_quad=40,base=None):
393     self.rc.raise_if_invalid(plan,base=base)
394     return sum(self._profit(plan,y,problem,mode,dist,n_quad) for y in YEARS)
395
396 def solve(self, baseline=None, problem=1, mode=1, dist='normal',
397     n_quad=40, seed=None, npop=NP, ngen=G, verbose=True,
398     k2_warm=False, mc_N=2000, mc_seed=0):
399     if seed is not None: self.seed=seed; self.rng=random.Random(seed)
400     self.mc_N=mc_N; self.mc_seed=mc_seed
401     self.mc_R = self.rev.corr_matrix() if problem==3 else None
402     self.mc_cache = {}
403     plan = {2023: baseline or {}}
404     base_cs = self.rc.crop_sets(plan[2023])
405     greedy = {}; prev = base_cs
406     for y in YEARS:
407         greedy[y] = self._greedy_year(prev,mode)
408         prev = self.rc.crop_sets(greedy[y])
409     prev = base_cs
410     for y in YEARS:
411         p2 = base_cs if y-2==2023 else (
412             self.rc.crop_sets(plan[y-2]) if y-2>=2024 else {})
413         vec0 = self._encode(greedy[y])

```

```

414         best,f = self._de_year(vec0,y,prev,p2,problem,mode,dist,n_quad,
415                                npop=npop,ngen=ngen)
416         plan[y] = self._decode(best,y,prev)
417         prev = self.rc.crop_sets(plan[y])
418         if mode==2:
419             self._repair_beans(plan)
420             prev = self.rc.crop_sets(plan[y])
421             if verbose: print(f' {y}: DE fitness = {f/1e4:.2f} wan')
422         self._repair_beans(plan)
423         self._polish(plan,problem,mode,dist,n_quad,max_iter=6)
424         self._repair_beans(plan)
425         full = integrate({y:plan[y] for y in YEARS},base=plan[2023])
426         self.rc.raise_if_invalid(full,base=plan[2023])
427         fit = sum(self._profit(plan,y,problem,mode,dist,n_quad) for y in YEARS)
428         return plan, fit
429
430 def load_2023_baseline(scheme=None):
431     sm = scheme or SchemeModel()
432     plant =
433         pd.read_csv(Path(__file__).resolve().parent.parent/'data'/'2023种植情况.csv',
434                     skipinitialspace=True)
435     base = defaultdict(lambda: defaultdict(list))
436     for _,r in plant.iterrows():
437         plot = str(r['种植地块']).strip()
438         s = 1 if str(r['种植季次']).strip() in ('单季','第一季') else 2
439         u = sm.unit_of(plot,s)
440         base[plot][s].append((int(r['作物编号']),
441                                float(r['种植面积/亩'])/sm.unit_area[u]))
441     return dict(base)

```